Bluestreak: Scaling DAG BFT by Sparsifying Metadata

Nikita Polyanskii
IOTA Foundation
nikitapolyansky@gmail.com

Ilya Vorobyev
IOTA Foundation
ilia.vorobev@iota.org

Sebastian Müller
Aix-Marseille Université
sebastian.muller@univ-amu.fr

Abstract

DAG-based Byzantine fault-tolerant (BFT) consensus protocols achieve high throughput by allowing many validators to propose concurrently, but scaling them to large committees remains challenging. In a committee of n validators, up to f of which may be Byzantine ($n = 3f+1$), dense round-based DAG designs require each block to reference at least $2f+1$ blocks from the previous round. This yields $O(n)$ metadata per block, $O(n^2)$ metadata per round, and $O(n^3)$ metadata bytes transmitted per round under all-to-all dissemination, increasing bandwidth and processing costs and making metadata, rather than payload, the latency bottleneck.

We present Bluestreak, a sparse uncertified DAG BFT consensus protocol that keeps non-leader blocks constant-size (in n) and concentrates committee-scale ancestry in a single leader block per round, yielding constant *average* metadata per block as committees grow. Bluestreak combines this sparse block format with a new leader commit rule co-designed for the sparse DAG and a new pull-based pacemaker, and we prove safety and liveness under partial synchrony using only collision-resistant hashes and standard digital signatures.

We implement and evaluate Bluestreak under wide-area latency spanning ten geo-distributed regions. Bluestreak scales from 10 to 400 validators on commodity 4-vCPU instances with sub-second WAN latency throughout (≈ 470 ms at n=10, ≈ 720 ms at n=400), keeping average per-block metadata constant at ≈ 320 bytes. At n=120, Bluestreak sustains ≈ 220k tx/s with LSM-tree storage and ≈ 400k tx/s with WAL-based storage, both at sub-second latency.

1 Introduction

DAG-based Byzantine fault-tolerant (BFT) consensus protocols have emerged as a prominent design point for high-throughput consensus under partial synchrony. We consider a committee of n validators, up to f of which may be Byzantine, where $n = 3f+1$. In these protocols all n validators propose blocks concurrently; each block references earlier blocks, forming a directed acyclic graph (DAG). Two main classes exist: *certified* DAGs such as Narwhal/Bullshark [9, 24], Sailfish [22], and Sailfish++ [23], in which each block is disseminated via reliable broadcast (RBC) before it can be referenced, and *uncertified* DAGs such as Cordial Miners [14] and Mysticeti [3], which use best-effort broadcast (BEB) for lower latency at the cost of handling equivocations in the commit rule. This design is already used in production: Mysticeti is the consensus protocol of Sui [15] and IOTA [13], and is used internally in Ika [21]; Sailfish [22] powers the decentralized Timeboost sequencer in the Espresso Network [11].

These protocols have a strong story for scaling transaction-throughput at a fixed committee, but scaling to large committees remains challenging. In dense round-based DAGs such as Bullshark and Mysticeti, each valid block carries at least $2f+1$ references to blocks from the previous round, so block metadata grows linearly with the committee. Each block already carries $O(\kappa n)$ bytes of consensus metadata, where κ is the security parameter, and even in the happy case the metadata communication of one round is $O(\kappa n^3)$ because n validators each broadcast such a block to n recipients. This paper primarily targets committee-size scaling by reducing this committee-scale metadata; because metadata competes directly with payload for bandwidth and processing, this also increases the transaction throughput a fixed committee can sustain.

Beyond bytes, dense metadata also increases CPU cost and can amplify latency during recovery. Each validator must hash-check $O(n)$ parent references per incoming block, traverse denser ancestry during ordering and catch-up, and, in pull-based designs, fetch missing parents before progress can resume. Under high load, faults, or desynchronization, these repair paths can dominate metadata traffic and, for dense DAG protocols, can reach $O(\kappa n^4)$ communication in the worst case [17]. Dense metadata therefore slows both axes of scaling at once: it reduces the maximum throughput attainable by a fixed committee and steepens latency growth as committees expand. Together, these costs create a *metadata wall* that caps practical validator-set sizes and weakens the decentralization story of DAG BFT.

Closest prior work attacks this problem in three main ways. *DAGs for the Masses* [1] sparsifies a certified Bullshark-like DAG by replacing the linear parent set of every block with a constant-size random sample from the previous round. Because safety then depends on the sample being unbiased, the protocol adds verifiable-sampling machinery, including vector commitments and sampling proofs. *Mysticeti-L* [17] instead keeps richer leader ancestry and sparsifies block references using multi-signatures (e.g., BLS [6]). *Autobahn*, *Angelfish*, and *Morpheus* [12, 16, 26] make metadata asymmetric in different ways. Autobahn keeps most dissemination objects in single-parent lanes and orders concise cuts of lane tips. Angelfish lets only a subset of validators create full DAG blocks with payload while the rest send lightweight votes. Morpheus avoids paying full DAG overhead at low load and escalates to an Autobahn-like high-throughput mode only when conflicts or load require it. Section 7 sharpens these

Protocol	Avg. e2e latency	Metadata per round	Happy-case communication	Multi-sig needed?
Bullshark [24]	8δ	$O(\kappa n^2)$	$O(\kappa n^3 + Mn)$	Yes
Sailfish++ [23]	6δ	$O(\kappa n^2)$	$O(\kappa n^3 + Mn)$	No
Mysticeti [3]	4.5δ	$O(\kappa n^2)$	$O(\kappa n^3 + Mn)$	No
Mysticeti-L [17]	4.5δ	$O(\kappa n)$	$O(\kappa n^2 + Mn)$	Yes
Bluestreak	4.5δ	$O(\kappa n)$	$O(\kappa n^2 + Mn)$	No

Table 1. Comparison of partially synchronous DAG-based BFT protocols with pull dissemination. Latency values are computed under the idealized conditions explained in Appendix D; happy-case communication values use the standard post-GST happy-case analysis from the same section. M: average total payload per round; n: number of nodes; κ: security parameter; δ: actual message delay. For Bullshark we assume reliable broadcast instantiated with multi-signatures as in Narwhal [9], where a block is certified in two communication rounds. Sailfish++ achieves optimistic two-round certification in a signature-free way [23]. Mysticeti-L uses multi-signatures (e.g., BLS [5, 6]).

comparisons and also places Bluestreak against the older sparse-link lineage of Hashgraph and the Tangle line.

Bluestreak. We present Bluestreak,[1] an uncertified sparse DAG BFT consensus protocol in which every block can carry transaction payload, but metadata is sparsified: non-leader blocks reference only up to two parents, while the single leader block per round carries the richer ancestry needed for safe commitment. Bluestreak achieves the same asymptotic split as Mysticeti-L—$O(\kappa)$ metadata in non-leader blocks and $O(\kappa n)$ in leader blocks—but uses only standard digital signatures and collision-resistant hashing, rather than multi-signatures, vector commitments, or sampling proofs. The key point is that sparsifying the block format is not enough by itself: the block format and commit rule must be co-designed with a pull-based pacemaker, which is central to our liveness proof. Together, these mechanisms yield a protocol that satisfies safety and liveness under partial synchrony with collision-resistant hashing and standard digital signatures alone.

This design reduces the metadata each validator processes per round from $O(\kappa n^2)$ to $O(\kappa n)$ while preserving the low-latency structure of Mysticeti and further reducing the already low computational overhead of uncertified DAGs. Our evaluation shows that this reduction sustains larger committees and improves achievable transaction throughput on fixed hardware. Figure 1 previews the committee-scaling results; the full methodology and setup appear in Section 8.

Contributions.

- We present Bluestreak, a sparse uncertified DAG BFT protocol with light non-leader blocks and richer leader blocks, reducing the metadata each node processes per round from $O(\kappa n^2)$ to $O(\kappa n)$ using only standard digital signatures and hashing.
- We give a commit rule for sparse DAGs and a pull-based pacemaker with formal synchronization guarantees after GST, and prove safety and liveness under partial synchrony.
- We derive the Bluestreak entries in Table 1: average end-to-end latency 4.5δ under idealized synchronization assumptions and happy-case communication $O(\kappa n^2 + Mn)$ under standard post-GST conditions, matching the best tradeoff in the table.
- We evaluate Bluestreak empirically under wide-area-style latency while varying committee size, offered load, and failures, showing that reduced metadata sustains larger committees and improves achievable transaction throughput compared to dense and BLS-based variants.
- We provide a shared Rust benchmarking codebase with comparable implementations and common networking, storage, serialization, transaction generation, and metrics, enabling apples-to-apples comparisons across DAG-based designs.

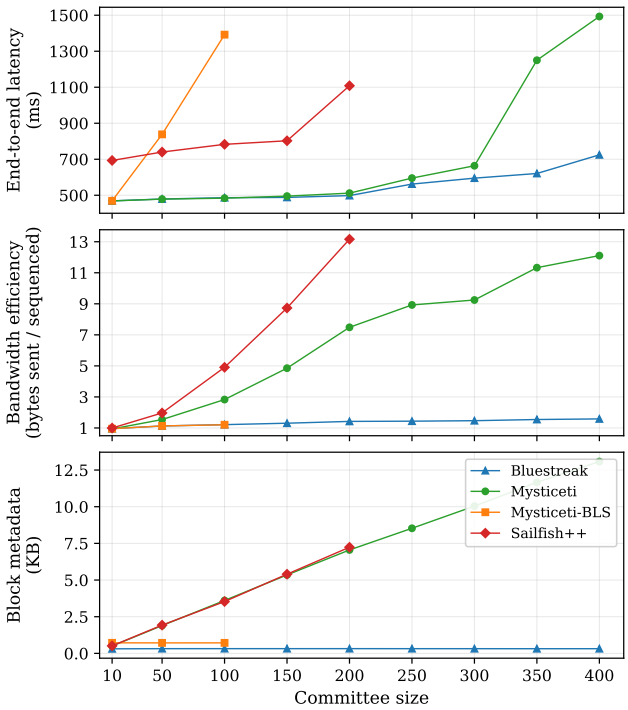

Figure 1. Committee scaling at 4 000 tx/s on c5d.xlarge instances (4 vCPUs, 8 GiB RAM) with RocksDB storage under synthetic wide-area latency (setup in Section 8). **Top:** median end-to-end transaction latency. **Middle:** bandwidth efficiency (bytes sent per sequenced byte). **Bottom:** average block metadata size.

Roadmap. Section 2 states the model and goals. Section 3 explains why Bluestreak separates metadata-light ordinary

[1]Following the convention that many DAG-based protocols are named after fish, *Bluestreak* refers to a cleaner fish that removes parasites from other fish. The protocol similarly "cleans" consensus metadata from dense DAG designs by shifting rich structure away from ordinary blocks.

blocks from metadata-heavy leader blocks. Appendix E.1 and sections 5 and 6 then define the sparse DAG, the leader-slot decision rule, and the pull pacemaker that together specify the protocol used in our evaluation. Section 7 positions Bluestreak against dense, sparse, and architecturally asymmetric DAG protocols. Section 8 presents the implementation and experiments, and Section 9 discusses limitations before Section 10. Extended implementation details, full proofs, the analysis behind the Table 1 latency and happy-case communication values, and complete protocol pseudocode appear in the Appendix.

2 Preliminaries

System Model. We consider a message-passing system $\mathcal{P} := \{p_1, \ldots, p_n\}$ consisting of $n = 3f + 1$ nodes. An adversary may control up to f nodes referred to as *Byzantine* or *faulty*. These nodes may deviate arbitrarily from the protocol. A node that follows the protocol throughout the execution is considered *correct* or *honest*.

We adopt the *partial synchrony* model of Dwork et al. [10]. Initially, the network may behave asynchronously, and an adversary can arbitrarily delay messages sent by honest nodes. After an unknown time called the *Global Stabilization Time* (GST), message delays become bounded: there exists an (unknown) delay bound δ such that all messages sent by honest nodes are delivered within δ. The protocol uses timeouts parameterized by a known bound Δ, and we assume $\delta \leq \Delta$ after GST.

Cryptographic Assumptions. We assume a computationally bounded adversary with security parameter κ. The protocol relies on two primitives: *collision-resistant hash functions* for block digests and hash references between blocks, with outputs of size κ bits; and a *Public Key Infrastructure (PKI)* using standard digital signatures for message authentication, where each node maintains a public-private key pair and signatures have size $O(\kappa)$. We write $(m)_p$ to denote signature by p on message m together with m.

State Machine Replication (SMR). Each honest node maintains an append-only output sequence of payloads. We say that the protocol implements BFT state machine replication if it satisfies the following two properties:

- **Safety (prefix consistency).** At any time, for any two honest nodes, one node's output sequence is a prefix of the other's.
- **Liveness.** If an honest node creates a block carrying payload m, then every honest node eventually appends m to its output sequence.

These properties are sufficient for BFT state machine replication: if nodes interpret payloads as transactions and apply the same deterministic state transition function to the output sequence, then all honest nodes execute the same sequence and remain consistent.

3 Design Rationale

Many DAG BFT protocols use *dense* references: almost every block points to the latest known blocks of (almost) every other node, so a single block carries $\Theta(n)$ hash references. This dense parent set is convenient because the same references serve four roles, corresponding to our four goals: (i) deterministic traversal for consistent sequencing once a leader is committed, (ii) collecting votes for that leader, (iii) creating or validating leader certificates (often implicitly, via verifiable ancestry patterns in uncertified DAGs), and (iv) pacing/spam resistance by forcing proposers to stay close to the committee. The downside is that it makes ordinary blocks expensive.

Bluestreak keeps these goals but assigns them asymmetrically across two block roles. Ordinary non-leader blocks are sparse (at most two references), while the single leader block of each round carries the richer ancestry needed for traversal and later decisions. The end-to-end effect of this split on committee scaling is shown in Figure 1 and evaluated in Section 8.

Goal 1: Deterministic traversal and sequencing. Once a leader is committed, nodes derive the same output order by running a deterministic traversal (graph search) over the leader's causal history. Bluestreak moves most cross-node references into leader blocks: non-leader blocks form per-node chains and optionally reference the previous-round leader, while the leader block of each round must include at least $2f+1$ references from the previous round and references the latest not-yet-referenced block of each other node.

Goal 2: Collect votes for the leader. Blocks vote for the previous-round leader by referencing it; omitting the reference is a non-vote. To give the leader time to propagate, honest validators wait up to a bounded timeout before creating their block (Section 6); if the leader block arrives in time it is referenced as a vote, otherwise the block is created without the reference. Because non-leader blocks may still reference the previous-round leader, voting works in essentially the same way as in dense round-based DAGs.

Goal 3: Create certificates for the leader. In uncertified DAG designs, references are used to create certificates for leaders implicitly in the DAG structure: block B is considered to be a certificate for some leader L if a certain pattern is present in the causal history of block B. Every node that knows the history of block B can verify that it is indeed a certificate for leader L. In that sense, such certificates are proved by the DAG structure itself.

Bluestreak uses a different approach. After observing $2f+1$ votes for a leader, a node writes an explicit *certificate claim* in a later block. Unlike an implicit DAG certificate, a claim is just a statement: it becomes meaningful only once the recipient has also received the voting blocks that justify it. For example, in Figure 2 block B_{r+2} carries a certificate claim

for leader A_r even though its causal history contains only two of the $2f+1 = 3$ votes. This motivates a key ingredient of Bluestreak: the local notion of *referenceable* blocks. Informally, a block is referenceable at a node once every certificate claim appearing in the block's causal history is already locally verifiable from the node's current DAG view; until then the block may be stored but is not used as a parent. Only referenceable blocks may be used as hash references by newly created blocks. This prevents the DAG from "carrying" unverified claims and is crucial for both the commit rule and the pull-based pacemaker. Appendix E.1 formalizes referenceable blocks.

Goal 4: Pacing and spam resistance. DAGs also use references as a pacing mechanism: if a block is supposed to point to many latest blocks, then a node must stay close to the rest of the network to keep creating valid blocks.

Rather than forcing each non-leader block to reference many latest blocks, Bluestreak paces honest progress via the pull pacemaker. By condition A1 in Section 6, an honest node advances to round r only after its local DAG contains $2f+1$ *referenceable* round-$(r-1)$ blocks from distinct nodes.

Byzantine nodes may still flood blocks from future rounds, but they cannot force honest nodes to advance: blocks from rounds an honest node has not reached can remain buffered, and non-leader blocks carry only $O(\kappa)$ block metadata. Concretely, each node is a leader once every n rounds, so a single Byzantine node can force an honest node to buffer at most n blocks before one of them is a leader block. That leader block contains up to n ancestor references, so the total buffered metadata per Byzantine node is $O(\kappa n)$—the same per-node overhead as a single round of an uncompressed dense DAG.

Commit-rule pipeline. Leader-slot decisions follow a simple pipeline (formalized in Section 5). In round r, the leader proposes a leader block. In round $r+1$, ordinary blocks either vote for that leader by referencing it or effectively cast a non-vote by omitting the reference. In round $r+2$, nodes that have seen $2f+1$ votes carry a certificate claim for the round-r leader. If $2f+1$ such claims are visible, the leader directly commits; if $2f+1$ non-votes are visible, the slot directly skips. If neither direct evidence is available, a later committed leader serves as an anchor: if its history contains a certificate claim for the earlier slot, that slot indirectly commits; otherwise it is skipped. Figures 2 and 3 illustrate the direct commit and direct skip cases on small DAG fragments. The indirect cases are shown later in Section 5. Once a leader is committed, all payloads in its causal history that have not yet been sequenced are appended to the output; skipped leaders contribute nothing and the protocol proceeds to the next undecided slot (Figure 4).

4 Sparse DAG Model

We present the sparse uncertified DAG model. Unlike prior uncertified DAG protocols where almost every block pays

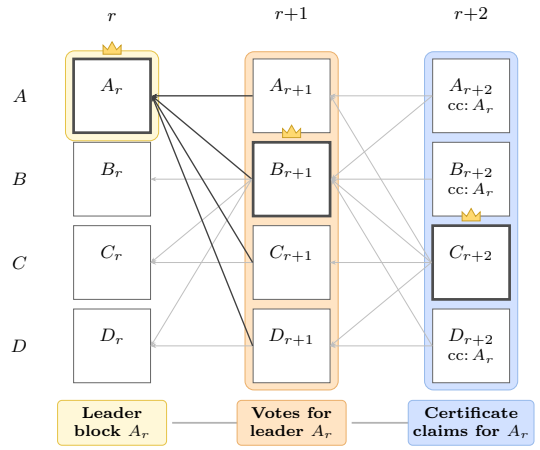

Figure 2. Direct commit in Bluestreak. Each non-leader block carries at most two references: to its own previous block and, optionally, to the previous-round leader. The figure shows only the round-$(r+2)$ certificate claims (cc) for the round-r leader, omitting claims for earlier leaders that may also appear in rounds r and $r+1$. The round-r leader directly commits once $2f+1$ such claims are visible.

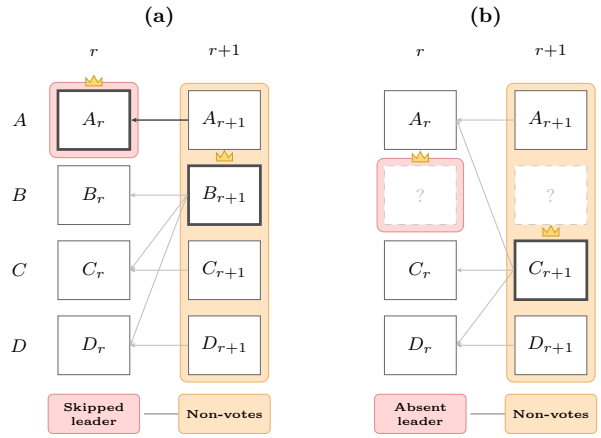

Figure 3. Direct skip in Bluestreak. (a) Leader block present but receives fewer than $2f+1$ votes. (b) Leader block never received; all round-$(r+1)$ blocks are non-votes. In both cases, $2f+1$ non-votes trigger a direct skip.

for node-scale ancestry, Bluestreak makes ordinary blocks cheap and lets the single leader block of each round carry the richer structure needed for traversal and later indirect decisions. This is the natural place to concentrate metadata: paying $O(\kappa n)$ once per round in a leader block is far cheaper than paying it in almost every one of the n blocks of the round. As in other uncertified DAGs, Byzantine nodes may still equivocate, so the commit rules must ensure that all

honest nodes reach consistent decisions despite conflicting blocks.

Block DAG and Rounds. Nodes concurrently create blocks in rounds starting with round 1. The n blocks of round 0 are predefined and known to all nodes. All blocks form a directed acyclic graph (DAG), also called a block DAG, where each block serves as a unique vertex, and a directed edge from block B to block B' indicates that B explicitly references B'. Every node creates a block, signs it, and multicasts it.

Leader Block and Scheduler. Nodes agree beforehand on a round-robin leader scheduler: node p_i proposes leader blocks in rounds $r = i, i + n, i + 2n \ldots$. Leader blocks play a special role in block creation, commit rules, and DAG traversal.

Block Format. Each block of node p_i is signed. The structure depends on whether p_i is the leader of the current round:

Non-leader blocks. For a non-leader node p_i in round r, the block has the format

$$B = (r,\ \{\texttt{self},\ \texttt{leader}_{r-1}?\},\ \texttt{certClaim},\ \texttt{payload})_{p_i},$$

where `self` is a hash reference to p_i's own block from round $r - 1$; $\texttt{leader}_{r-1}$ is an optional hash reference to the leader block of round $r - 1$, included only if that leader block is available and *referenceable* in the author's local DAG at block-creation time; `certClaim` is either the hash of the leader block from round $r - 2$ that node p_i claims is certified, or $\perp$ if p_i has not observed a certificate pattern for that slot (see Section 5); and `payload` is the transaction payload. Non-leader blocks have metadata size $O(\kappa)$.

Leader blocks. For the leader node p_i in round r, the block has the format

$$B = (r,\ \texttt{ancestors},\ \texttt{payload})_{p_i},$$

where `ancestors` contains at most one hash reference per node. The leader must include in `ancestors` a hash reference to its own block from round $r - 1$; this self reference is a mandatory ordinary element of `ancestors`, not a separate field. For each node p_j, the leader should include a hash reference to the latest (highest-round) *referenceable* block of p_j known locally at block-creation time whose round is at most $r-1$, provided that this block has not already been referenced earlier by p_i. If multiple such blocks exist in that latest round (e.g., due to equivocation), the leader picks one. If no such block exists, the leader includes no reference for p_j. Consequently, `ancestors` may contain fewer than n references. The leader must also ensure that `ancestors` contains at least $2f + 1$ references to round-$(r - 1)$ blocks. Receivers validate a leader block by checking these structural requirements on `ancestors` (at most one reference per node, the mandatory self reference, and the $2f + 1$ round-$(r - 1)$ references); in particular, whether the leader indeed referenced the latest blocks known to it is not externally verifiable. We call a leader block *valid* if it satisfies the structural requirements. Leader blocks therefore have metadata size $O(\kappa n)$.

Local DAG construction. Each node p_i maintains its own local DAG DAG_i of blocks it has added. For a block B, let $\mathsf{hist}(B)$ denote its causal history, namely the set consisting of B itself together with all blocks transitively reachable from B via hash references.

When p_i receives a block B, it first checks the signature and the block-format conditions that can be tested immediately. In particular, a non-leader block must reference its author's own block from round $r - 1$, and a leader block must satisfy the structural requirements on `ancestors`. Blocks that fail validation are discarded and never added to the local DAG.

A node adds a block B to DAG_i only after adding every block that B directly references. For non-leader blocks, these references are the blocks named in `self` and the optional leader field. For leader blocks, these references are the blocks named in `ancestors`. If some referenced block is missing, node p_i keeps B in a pending buffer until the missing history arrives, either via normal dissemination or via the pull-based recovery described in Section 6. Therefore, whenever B is in DAG_i, its entire causal history $\mathsf{hist}(B)$ is already present in DAG_i.

Only blocks in the local DAG may contribute to votes, certificate patterns, or round advancement conditions. However, newly created blocks are allowed to reference only a stricter subset of blocks in the local DAG, called *referenceable* blocks.

Referenceable blocks. Fix a node p_i and its current local DAG DAG_i. A block X in DAG_i is *referenceable* at p_i if every certificate claim appearing in $\mathsf{hist}(X)$ is already locally provable from blocks in DAG_i. That is, whenever some block in $\mathsf{hist}(X)$ carries $\texttt{certClaim} = h(L)$ for a leader block L from slot $(\cdot, r)$, the local DAG DAG_i already contains $2f + 1$ round-$(r + 1)$ voting blocks from distinct nodes for L. A block in the local DAG may therefore be temporarily non-referenceable until the necessary voting blocks arrive. Only referenceable blocks may be used as future hash references by newly created blocks.

Leader Slot. A *leader slot* is a pair (p_i, r), where p_i is the leader in round r, $i = ((r-1) \bmod n)+1$. Each slot is initially in the state UNDECIDED. By interpreting the DAG structure, each node attempts to make a decision COMMIT or SKIP for every leader slot.

Voting. We say that a block B from round $r + 1$ *votes* for a leader block L from round r if B references L. For a non-leader block, this means that its optional field $\texttt{leader}_r$ points to L; for a leader block, this means that L appears in `ancestors`. A round-$(r + 1)$ block is a *non-vote* for a proposal L from slot (p_i, r) if it does not reference L. Thus, a non-leader block with no leader reference is a non-vote for every proposal in that slot, and any block that references a different leader block from the same slot is also a non-vote for L.

Certificate Pattern for Leader Slot. To make decisions, nodes use the following pattern in their local DAGs: blocks from at least $2f + 1$ nodes at round $r + 1$ reference the same block B of round r created by leader p_i. We then say that B is *certified.*

5 Commit Rule for Leader Slots

In the sparse DAG, every block from round $r + 2$ may make a certificate claim about a leader block from round r. Non-leader blocks carry this claim explicitly in `certClaim`. Leader blocks carry the same claim implicitly: the claim is justified structurally when their ancestors already contain $2f + 1$ round-$(r + 1)$ votes for that leader block.

Certificate claims and local provability.

Certificate claim. Consider a leader slot (p_i, r). A round-$(r + 2)$ block B makes a *certificate claim* for leader block L in slot (p_i, r) if, in the local view of B's author, $2f + 1$ round-$(r + 1)$ blocks vote for L. For a non-leader block, this claim is represented explicitly by the field `certClaim` $= h(L)$. For a leader block, this claim is represented implicitly by the presence of those $2f + 1$ round-$(r + 1)$ voting blocks in its ancestors. An honest node makes such a claim only if it has genuinely observed the certificate pattern for L. Byzantine nodes may claim arbitrarily.

Locally provable certificate claim. A certificate claim for slot (p_i, r) and leader block L is *locally provable* in a node's local DAG if that local DAG contains $2f + 1$ round-$(r + 1)$ blocks from distinct nodes that vote for L. This notion is local to the node's current DAG: a certificate claim may already appear in the causal history of some block in the local DAG while remaining non-referenceable until the corresponding voting blocks are also present. It is used to determine when a block becomes referenceable and may be used as a hash reference by newly created blocks.

Direct Commit Rule. A node marks a leader slot (p_i, r) as COMMIT and commits leader block L if its local DAG contains $2f + 1$ round-$(r + 2)$ blocks from distinct nodes that make a certificate claim for L.

Direct Skip Rule for Proposals. A node directly skips a proposal L in slot (p_i, r) if its local DAG contains $2f + 1$ round-$(r + 1)$ blocks from distinct nodes that are non-votes for L.

Direct Skip Rule for Slots. A node marks a leader slot (p_i, r) as SKIP once its local DAG contains at least $2f + 1$ round-$(r + 1)$ blocks from distinct nodes and every proposal from slot (p_i, r) present in its local DAG is directly skipped. This also covers unseen proposals: if the node's local DAG contains no leader block from slot (p_i, r), then observing $2f + 1$ round-$(r + 1)$ blocks already suffices to skip the slot, because each of those blocks is a non-vote for any proposal in that slot.

Indirect Decision Rule. For slots remaining UNDECIDED after the direct rules, nodes proceed from higher to lower rounds. For a leader slot (p_i, r), scan leader slots in increasing round order starting from round $r + 3$ and find the first slot whose state is not SKIP.

- If no such slot exists, or if the first such slot is still marked UNDECIDED, then (p_i, r) remains UNDECIDED.
- Otherwise, the first such slot is marked COMMIT ; call its committed leader block the *anchor* A, and let its round be r'.
- Since anchors are searched only inside the local DAG and a node adds a block only after also adding all blocks it references, the node can inspect $\mathsf{hist}(A)$ locally.
- *Indirect commit:* The node marks (p_i, r) as COMMIT and commits leader block L_r if $\mathsf{hist}(A)$ contains a certificate claim for L_r in slot (p_i, r).
- *Indirect skip:* Otherwise, the node marks (p_i, r) as SKIP. Equivalently, an honest node skips only when the anchor history contains no certificate claim for that slot.

Figure 4 shows an end-to-end eight-round execution in which Bluestreak exhibits all four decision outcomes: direct commit, indirect commit, indirect skip, and direct skip, before leaving the final slots undecided.

Correctness intuition. Safety and liveness come from the interaction of quorum intersections, referenceability, and delayed pull recovery. *Safety.* Direct decisions are quorum based: a direct commit for leader L requires $2f$+1 round-(r+2) claims for L, and every honest such claim is backed by $2f$+1 round-(r+1) votes for L. Since each honest node creates only one round-(r+1) block and can vote for at most one leader in the slot, quorum intersection rules out two conflicting certified leaders and also rules out a direct commit and a direct skip for the same slot at honest nodes. Indirect decisions inherit this uniqueness because honest nodes extend only *referenceable* blocks, and a block becomes referenceable only once every inherited certificate claim in its causal history is locally provable from explicit voting blocks. Thus if a committed anchor contains a claim for an earlier slot, honest votes for that anchor come from nodes that have already verified the claim, so anchor histories cannot justify conflicting earlier decisions. *Liveness.* After GST, honest nodes remain within bounded round skew. Whenever one honest node advances, its round-advance message certifies that $2f$+1 referenceable blocks for the previous round already exist somewhere in the honest system. If another honest node is missing history, delayed pull recovery returns a self-contained set of missing blocks and supporting votes sufficient to add those blocks to the local DAG and make them referenceable. This prevents deadlock on hidden certificate claims: lagging nodes can reconstruct enough of the sparse DAG to satisfy A1, create their own block to satisfy A2, and continue. Once honest nodes are roughly synchronized, each slot eventually gains direct evidence: an honest

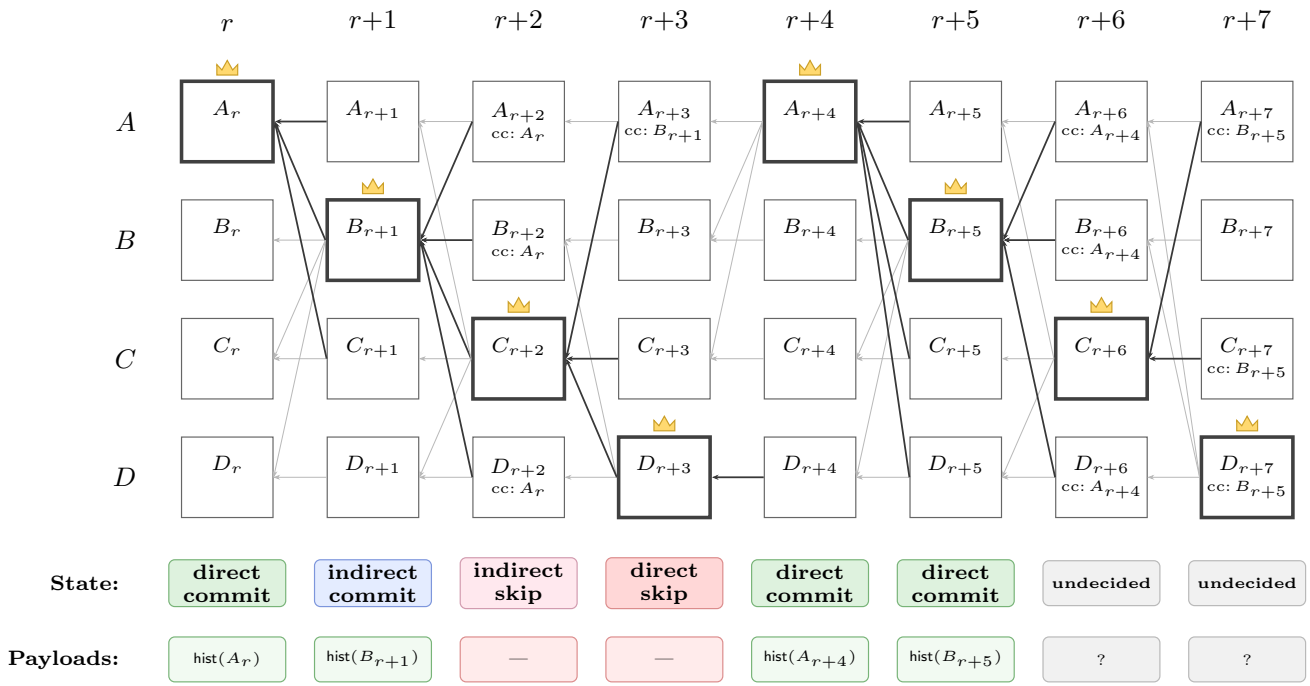

Figure 4. Example Bluestreak DAG for four nodes (A–D) over eight rounds with $n = 4$. Crowned nodes are leaders. Black arrows denote references to leaders, and gray arrows denote references to non-leader blocks. Labels of the form `cc:` mark blocks that make a certificate claim (cc) for the shown earlier leader. The *State* row shows the decision reached for each leader slot. The *Payloads* row lists $\mathsf{hist}(L)$ for the committed leaders whose histories are appended to the output at that point. Slot A_r is `direct commit`: round r+1 contains three votes for A_r, and round r+2 contains three certificate claims (A_{r+2}, B_{r+2}, D_{r+2}), so A_r is directly committed. For slot B_{r+1}, we cannot directly decide whether it should be committed or skipped: round r+2 contains four references to B_{r+1}. In round r+3, node A makes an explicit certificate claim for it in A_{r+3}, and leader block D_{r+3} carries one additional implicit claim. This is still fewer than the three claims needed for direct commit, so slot B_{r+1} remains undecided at that stage. For slot C_{r+2}, we also cannot directly decide whether it should be committed or skipped: round r+3 contains three references to C_{r+2} (A_{r+3}, C_{r+3}, and D_{r+3}), so the slot is not directly skipped. At the same time, no round-(r+4) block carries a certificate claim for C_{r+2}, so it is not directly committed either. Slot D_{r+3} is `direct skip`: in round r+4 only D_{r+4} references D_{r+3}, while A_{r+4}, B_{r+4}, and C_{r+4} reference only non-leader blocks, so the slot is directly skipped. Slots A_{r+4} and B_{r+5} are `direct commit`: each receives at least three votes in the next round and at least three certificate claims in the round after that. Once A_{r+4} is directly committed, it also serves as an anchor for the earlier slot B_{r+1}: its history contains A_{r+3}, which carries a certificate claim for B_{r+1}, so the indirect rule commits B_{r+1}. Thus slot B_{r+1} is `indirect commit`. The same anchor search applied to the earlier slot C_{r+2} reaches B_{r+5}, and its history contains no certificate claim for C_{r+2}, so slot C_{r+2} is `indirect skip`. Finally, slots C_{r+6} and D_{r+7} remain undecided in this figure: round r+7 contains only two votes for C_{r+6}, and there are no later rounds to resolve D_{r+7}.

leader is seen in time, gathers $2f+1$ votes, and is then directly committed after the next round of certificate claims. Faulty or delayed leaders may still be skipped and cost extra rounds, but they do not prevent future commits, and every payload created by an honest node eventually enters the history of a directly committed honest leader. The full proof appears in the Appendix.

Commit sequence and output ordering. After applying the decision rules above, each node derives an ordered sequence of committed leader blocks by scanning leader slots in increasing round order. It appends the committed leader block of every slot marked COMMIT, ignores slots marked SKIP, and stops at the first slot marked UNDECIDED.

To linearize blocks and define the output sequence, we fix a deterministic topological ordering sort over blocks consistent with hash references. For a block B, let $\mathsf{round}(B)$ and $\mathsf{author}(B) \in \{1, \ldots, n\}$ denote its round number and creator.

Nodes process committed leaders in increasing slot order. When a leader block L becomes committed, the node appends the payloads of blocks that become newly reachable through L: the blocks in $\mathsf{hist}(L)$ that are not in the causal history of any previously committed leader in the sequence. It lists these newly reachable blocks in the order sort and appends their payloads. If several blocks share the same pair (author, round) (equivocation), the node appends the payload of the first such block in sort and ignores the rest. This yields a deterministic total order of appended payloads; in

particular, at most one block per node per round contributes to the output.

6 Pull Pacemaker

The pull pacemaker has three parts: round advancement, block creation, and delayed pull recovery. The conditions are summarized in Table 2. We use two delay parameters, δ_D and δ_A, for block-triggered and advance-triggered pull recovery, respectively; throughout this section we set $\delta_A = \delta_D = \Delta$.

Round advancement. A node advances from round $r-1$ to round r once two conditions hold: (A1) its local DAG contains $2f+1$ *referenceable* round-$(r-1)$ blocks from distinct nodes, and (A2) it has created its own block of round $r-1$. Condition A2 prevents nodes from advancing without contributing the round-$(r-1)$ blocks that later serve as votes, non-votes, and certificate-claim carriers. Condition A1 ensures that advancement happens only when enough referenceable support from the previous round is already available.

Whenever a node advances to round r, it immediately broadcasts a small round-advance message $(\mathsf{advance}, r)$ (B1).

Block creation. After advancing to round r, a node creates its round-r block once at least one of the following holds: (C1) its local DAG contains a *referenceable* round-$(r-1)$ leader block (L1) and either $2f+1$ *referenceable* round-$(r-1)$ votes for the leader of round $r-2$ or enough round-$(r-1)$ non-votes to directly skip slot $r-2$ (L2); (C2) its round-r timeout expires, where $\delta_{LT} = 8\Delta$; or (C3) its local DAG contains $2f+1$ *referenceable* round-r blocks from distinct nodes. If the creator is the leader of round r and C1 holds via the vote option in L2, then it includes in its ancestor list those $2f+1$ round-$(r-1)$ voting blocks, pinning down which round-$(r-2)$ leader is being certified even if some nodes equivocate.

Whenever a node creates its round-r block, it immediately broadcasts that block to all nodes (B2).

Delayed pull recovery. In the network, blocks and round-advance messages may be delayed and arrive out of order. A node may therefore receive a block before the referenced history needed to add it to the local DAG, before the voting blocks needed to justify inherited certificate claims, or may learn that others advanced to round r before it has enough data to do so. The pacemaker therefore treats pull as a repair path. Whenever a node receives (i) a block B whose prerequisites are not yet satisfied, or (ii) a round-advance message $(\mathsf{advance}, r)$ while it is still below round r, it starts a pull-delay timer (P1): δ_D for blocks and δ_A for round-advance messages.

If missing data has not arrived by timer expiry, the node actively pulls the missing prerequisites (P2), requesting them from the sender of the triggering block or round-advance message. Since causal histories can be large, the sender need not resend all blocks up to round 0: it responds with the requested blocks together with the missing referenced history (transitively), omitting any blocks that it can infer the receiver already has from prior communication. Concretely, if the sender has previously received from the receiver a block that references some block X, then the sender can omit X from the response, since honest nodes create blocks only after adding their referenced history. For a block B, these prerequisites are (a) referenced causal-history blocks needed before B can be added to the local DAG, and (b) voting blocks needed to make every inherited certificate claim in $\mathsf{hist}(B)$ locally provable and thus make B referenceable. For an advance message $(\mathsf{advance}, r)$, the response contains the data needed to advance up to round r: for each missing round $k < r$, it contains a set S_k of $2f+1$ referenceable round-k blocks from distinct nodes (enough to satisfy A1; e.g., the blocks that allowed the sender to satisfy A1 and advance), together with all blocks (including voting blocks) that the receiver is missing in order to (i) add every block in S_k to the local DAG (including all referenced history, transitively) and (ii) make every block in S_k referenceable. The response is self-contained: it includes all missing blocks referenced by the returned blocks (transitively), so one request/response exchange suffices. This lets a lagging node create its own missing blocks round by round (using C3) and then advance using A1 and A2.

7 Related Work

The sparse-link intuition predates current round-based BFT DAGs. Hashgraph [4] already used a constant-degree uncertified gossip DAG, and the Tangle line [18, 19] explored low-degree attachment together with random tip selection and later auxiliary finality mechanisms. These systems made the central tradeoff visible early: fewer explicit edges lower per-object communication, but confirmation and even simple local traversal then depend more on transitive reachability, probabilistic selection rules, or auxiliary voting / finality mechanisms. They are therefore important ancestors of sparse DAG thinking, but they do not directly solve the partially synchronous node-set BFT problem considered here. For a broader survey of DAG-based consensus protocols, see the recent SoK [20]; here we focus only on work most relevant to metadata placement.

Among modern BFT DAG protocols, Narwhal / Bullshark and their descendants [9, 23, 24] are the dense baseline: rich previous-round ancestry makes reachability and DAG traversal easy for local commit rules, but it forces node-scale metadata into nearly every block. Cordial Miners and Mysticeti [3, 14] remove RBC certificates and cut the control path, yet they still retain dense or otherwise nontrivial ancestry costs.

The closest modern sparse-metadata lines differ in where they concentrate the heavy structure. DAGs for the Masses [1] sparsifies every block via random sampled references and

Advance round $(r-1) \rightarrow r$	**Create block of round** r	**Broadcast / pull**
A1: local DAG contains $2f+1$ *referenceable* blocks of round $r-1$ from distinct nodes AND **A2:** created block of round $r-1$	**C1:** local DAG satisfies: **L1:** local DAG contains a *referenceable* leader block of round $r-1$ AND **L2:** local DAG contains either $2f+1$ *referenceable* round-$(r-1)$ votes for the leader of round $r-2$, or enough round-$(r-1)$ non-votes to directly skip slot $r-2$ OR **C2:** expired timeout $\delta_{LT} = 8\Delta$ OR **C3:** local DAG contains $2f+1$ *referenceable* round-r blocks from distinct nodes	**B1:** on round advance, broadcast $(\text{advance}, r)$ **B2:** on block creation, broadcast the new block **P1:** on receiving a block or $(\text{advance}, r)$ while still behind, start a pull-delay timer **P2:** on pull-delay expiry, pull missing prerequisites

Table 2. Pull-pacemaker conditions.

additional sampling machinery in a Bullshark-like certified DAG. Autobahn, Angelfish, and Morpheus [12, 16, 26] are more asymmetric: they keep many ordinary objects or ordinary participants cheap, while richer global structure lives in cuts, special vertices, or only the high-throughput regime. Mysticeti-L [17] is the closest exact structural comparison to Bluestreak: ordinary blocks become very light while leader blocks retain the richer ancestry needed for reachability and commitment, but Mysticeti-L relies on multi-signature compression.

Bluestreak occupies a narrower point in this design space: an uncertified sparse DAG with light ordinary blocks, rich leader blocks, and a commit path that requires only standard digital signatures. The claim of this paper is therefore not that sparse links are new, but that this particular tradeoff point—deterministic leader-based ordering with sparse ordinary ancestry, explicit reachability through special blocks, and no BLS or KZG machinery—is worth isolating and evaluating empirically.

8 Implementation and Evaluation

We implement all four protocols—Mysticeti, Mysticeti-BLS, Bluestreak, and Sailfish++—in a single Rust codebase. They share identical networking, storage, serialization, transaction generation, and metrics infrastructure; only the consensus logic and block format differ. This eliminates confounding variables from differing I/O stacks, language runtimes, or serialization formats. Starting from a fork of the open-source Mysticeti implementation,[2] we implemented Bluestreak as well as Mysticeti-BLS—an instance of the Mysticeti-L protocol described in [17] that replaces per-validator vote lists with BLS12-381 aggregate signatures. The Sailfish++ consensus logic was ported from the authors' open-source implementation [23],[3] All protocols were reimplemented on top of our shared infrastructure to ensure a fair comparison.[4] Additional implementation details, protocol-specific notes, and the full RTT matrix are listed in the Appendix.

Common setup. All protocols are configured with a single leader per round and a leader timeout of 400 ms. All experiments deploy nodes in a single AWS region (eu-west-1) and inject synthetic wide-area latency. Each node is assigned round-robin to one of ten virtual regions modelling us-east-1, us-west-1, ca-central-1, eu-west-1, eu-south-1, eu-north-1, sa-east-1, ap-south-1, ap-southeast-1, and ap-northeast-1. The one-way delay between two nodes equals half the measured round-trip time between their virtual regions. Round-trip times range from 1 ms (intra-region) to 309 ms (eu-west-1–eu-south-1); the average one-way latency experienced by a single node to all its peers ranges from 57 ms (us-east-1, us-west-1) to 105 ms (eu-west-1, eu-south-1), depending on placement. The full 10×10 RTT matrix appears in the Appendix. Transactions are 512-byte payloads: an 8-byte submission timestamp followed by random bytes (almost incompressible by the storage engine). Transactions are generated at a continuous pace by a client attached to each validator; the aggregate load is split equally among all nodes. All protocols use pull-based dissemination. Unless stated otherwise, blocks are persisted in RocksDB; Q5 replaces RocksDB with the WAL-based Tidehunter engine [7]. Each run lasts at least 180 s after reaching steady state. End-to-end latency measures the time from when a transaction arrives at the proposing node until it is sequenced by honest nodes.

We ask the following questions:

- **Q1.** *How does each protocol's latency, bandwidth overhead, and metadata size scale as the committee grows on fixed hardware?*
- **Q2.** *What is the maximum throughput each protocol sustains at 120 nodes before latency degrades?*
- **Q3.** *Do latency, bandwidth, and CPU remain stable under sustained moderate load over an extended run?*
- **Q4.** *How does latency behave when one-third of the nodes crash permanently?*

[2] https://github.com/MystenLabs/mysticeti
[3] https://github.com/qyu100/SFSailfish, branch `OptSFSailfish`.
[4] Our implementation is available at https://github.com/iotaledger/starfish.

Q5. *How does replacing LSM-tree storage with a WAL-based engine affect peak throughput and storage footprint?*

Q1: Committee scaling. We deploy on c5d.xlarge instances (4 vCPUs, 8 GiB RAM, up to 10 Gbps, NVMe SSD) and vary the committee size over $n \in \{10, 50, 100, \ldots, 400\}$ at a fixed aggregate load of 4 000 tx/s using RocksDB storage. We report three metrics (Figure 1): median end-to-end transaction latency (p50), bandwidth efficiency (total bytes sent per sequenced byte), and average block metadata size. Mysticeti-BLS is limited to $n \leq 100$ because BLS verification saturates the available CPU beyond that point; Sailfish++ is limited to $n \leq 200$.

The results (Figure 1) confirm the theoretical advantage of sparse DAG metadata. Bluestreak matches Mysticeti's latency at small committees and stays below it from $n = 150$ onward, reaching 400 nodes with only a modest latency increase. Mysticeti's latency, by contrast, rises sharply beyond $n = 300$ as its $O(n)$-sized blocks impose growing processing and communication overhead on every node. The bandwidth efficiency panel makes the gap explicit: Mysticeti's cost per sequenced byte grows linearly with n, exceeding 12× at $n = 400$, while Bluestreak remains nearly flat between 1× and 2× across the entire range—a direct consequence of its $O(1)$ metadata per non-leader block. The bottom panel shows the root cause: Mysticeti's block metadata grows to 13 KB at $n = 400$, whereas Bluestreak's metadata remains around 320 bytes regardless of committee size. The concrete happy-case estimate in Appendix Dyields an amortized metadata cost of about 284 bytes per block, close to the measured 320 bytes. The remaining gap comes from constant-size message-format overhead in the implementation, such as wrapper and serialization fields, which the abstract analysis does not count.

Q2: Throughput–latency tradeoff. We fix the committee at $n = 120$ on c5d.4xlarge instances (16 vCPUs, 32 GiB RAM, up to 10 Gbps) and increase the aggregate transaction load from 1 000 to 300 000 tx/s using RocksDB storage. Figure 5 plots the observed throughput against median end-to-end latency (p50), with p25–p75 bars. We also deployed Bluestreak across actual geo-distributed regions and confirmed that the measured end-to-end latency matches the synthetic WAN results within 10%.

All four protocols sustain over 190 000 tx/s at sub-second latency. The top panel of Figure 5 shows the DAG round duration: the three uncertified protocols (Mysticeti, Mysticeti-BLS, Bluestreak) complete a round in ≈ 100–110 ms at low load (≈ 9–10 rounds/s), while Sailfish++—the only certified-DAG protocol in our comparison—takes ≈ 220 ms per round (≈ 4.5 rounds/s), because each Sailfish++ block must collect echo messages from at least $5n/6$ nodes before it is optimistically certified, effectively doubling the per-round duration compared to uncertified protocols.

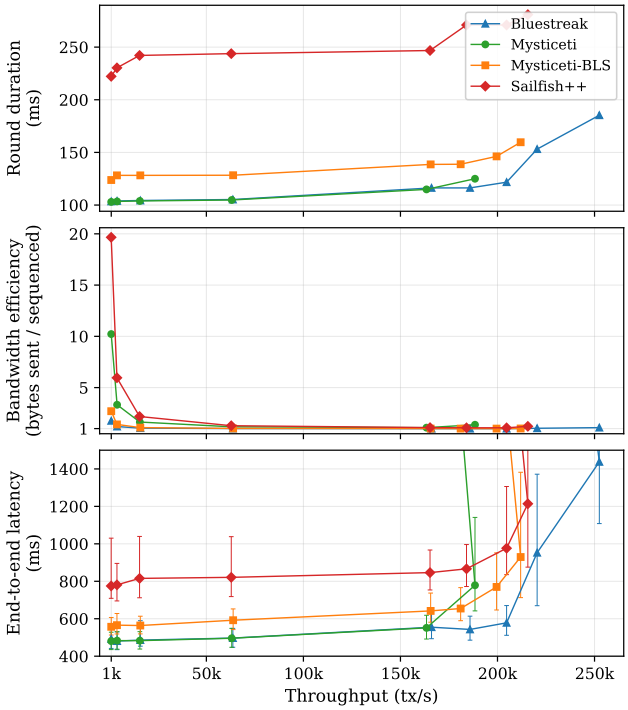

Figure 5. Throughput scaling at $n = 120$ on c5d.4xlarge instances (16 vCPUs, 32 GiB RAM) with RocksDB storage and synthetic WAN latency. **Top:** DAG round duration. **Middle:** bandwidth efficiency (bytes sent per sequenced byte). **Bottom:** median end-to-end latency (p25–p75 bars).

Bluestreak reaches the highest throughput (≈ 220k tx/s) before latency rises, matching Mysticeti's round rate while carrying far less metadata per block. Mysticeti saturates earlier (≈ 190k tx/s) because each block carries $O(n)$ parent references: a block cannot be integrated into the local DAG until all its ancestors are present, and under high load nodes increasingly fall behind, triggering missing-ancestor requests that further slow progress. Mysticeti-BLS tracks close to Bluestreak up to ≈ 200k tx/s but degrades faster because BLS aggregate verification, even with a dedicated worker pool, saturates the available 16 vCPUs at high block rates; on larger instances this gap would narrow. Sailfish++ operates at a consistently higher baseline latency (≈ 780 ms) due to its certification overhead, yet compensates with larger blocks per round and achieves comparable peak throughput. The middle panel confirms the bandwidth picture at this committee size: Bluestreak's average block metadata is ≈ 320 bytes, Mysticeti-BLS ≈ 730 bytes (constant thanks to BLS aggregation), while Mysticeti and Sailfish++ both carry ≈ 4.3 KB of per-block metadata that grows linearly with n.

Q3: Stability under sustained load. We run Bluestreak and Mysticeti with $n = 100$ nodes on c5d.xlarge instances at a sustained aggregate load of 20 000 tx/s using RocksDB

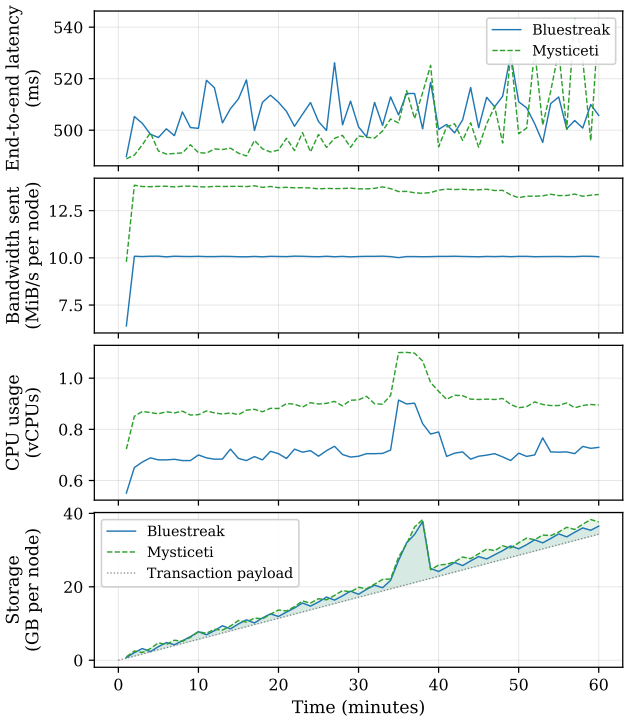

Figure 6. Stability at $n = 100$, load 20 000 tx/s on c5d.xlarge instances (4 vCPUs, 8 GiB RAM) with RocksDB storage. **Top to bottom:** median end-to-end latency, per-node sent bandwidth, per-node CPU usage, and per-node storage footprint.

storage and sample latency, bandwidth, and CPU usage every minute (Figure 6).

Both protocols maintain flat end-to-end latency around 500 ms throughout the run, with no upward drift. Bluestreak sends ≈ 10.1 MiB/s per node versus Mysticeti's ≈ 13.7 MiB/s—a 30% reduction in outbound wire traffic. The aggregate payload rate is $20\,000 \times 512$ B ≈ 9.8 MiB/s; under pull-based dissemination each transaction traverses the network once, so the per-node payload contribution is ≈ 9.7 MiB/s. The remaining bandwidth is protocol metadata: ≈ 0.4 MiB/s per node for Bluestreak (4% of total) versus ≈ 4 MiB/s for Mysticeti (30%). CPU usage is ≈ 0.7 vCPUs for Bluestreak and ≈ 0.9 vCPUs for Mysticeti; the brief spike near minute 35 is caused by RocksDB background compaction, not the consensus protocol.

Q4: Crash-fault tolerance. We use the same setup as Q3 (RocksDB; $n = 100$, 20 000 tx/s on c5d.xlarge) and permanently stop the first 33 nodes (authorities 0–32)—the maximum Byzantine threshold $f = \lfloor (n-1)/3 \rfloor$—at the 120-second mark (Figure 7). The remaining 67 nodes continue to operate.

Before the outage, both Bluestreak and Mysticeti maintain a steady-state latency of ≈ 490 ms. After one-third of the committee crashes, both protocols exhibit a periodic commit pattern: during commit phases the median latency across

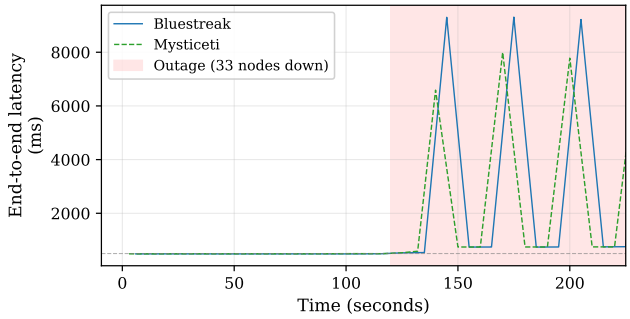

Figure 7. Crash-fault tolerance at $n = 100$, 20 000 tx/s with RocksDB storage. At $t = 120$ s, 33 nodes are permanently stopped. The shaded region marks the outage period.

active nodes rises to ≈ 740 ms, while between bursts the accumulated backlog drives the reported p50 above 7–9 s. The ≈ 30 s oscillation cycle reflects the protocol committing a batch of rounds, then waiting for the DAG to rebuild sufficient structure among the surviving 67 nodes. Despite the oscillation, neither protocol stalls—both continue to commit transactions throughout the outage. The baseline latency increase from 490 to 740 ms is expected: with fewer nodes the DAG takes longer to accumulate the $2f+1$ blocks needed per round, and the remaining nodes absorb the full transaction load. Part of the latency increase is also due to the fixed round-robin leader schedule: crashed nodes still occupy their designated leader slots, and the protocol must wait for a leader timeout before skipping to the next round. Leader reputation mechanisms—formalized by Carousel [8] and adopted for DAG-BFT by Shoal++ [2] and Hammerhead [25]—can dynamically exclude unresponsive leaders and reduce the number of wasted rounds. Integrating such a subprotocol is orthogonal to the core commit rule and is left as future work.

Q5: WAL-based storage. In previous Q1–Q4 every node stores blocks in RocksDB, an LSM-tree engine whose write amplification and background compaction create a storage-IO bottleneck that equally throttles all protocols. To isolate the effect of per-block metadata size on throughput, we replace RocksDB with *Tidehunter* [7], a write-ahead-log (WAL) engine that appends blocks sequentially and avoids compaction entirely.

Figure 8 shows the throughput–latency tradeoff at $n = 120$. Bluestreak sustains ≈ 420k tx/s before latency exceeds 1.5 s, whereas Mysticeti saturates around 190k tx/s—a 2.2× difference that was invisible under RocksDB (Figure 5).

The stability test (Figure 9) runs both protocols at n=400 with 4 000 tx/s on c5d.xlarge instances using Tidehunter

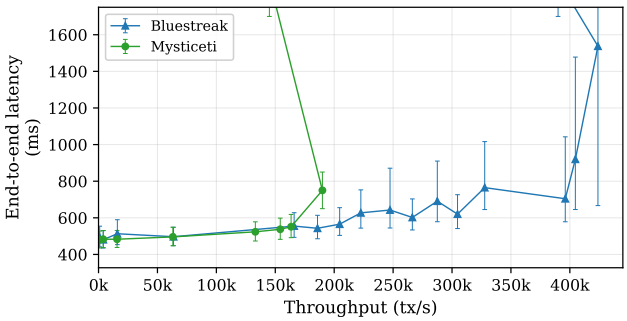

Figure 8. Throughput vs. latency at $n = 120$ on m5d.4xlarge instances (16 vCPUs, 64 GiB RAM), using WAL-based storage (Tidehunter). Bluestreak sustains 400k tx/s with sub-second latency; Mysticeti saturates near 190k tx/s.

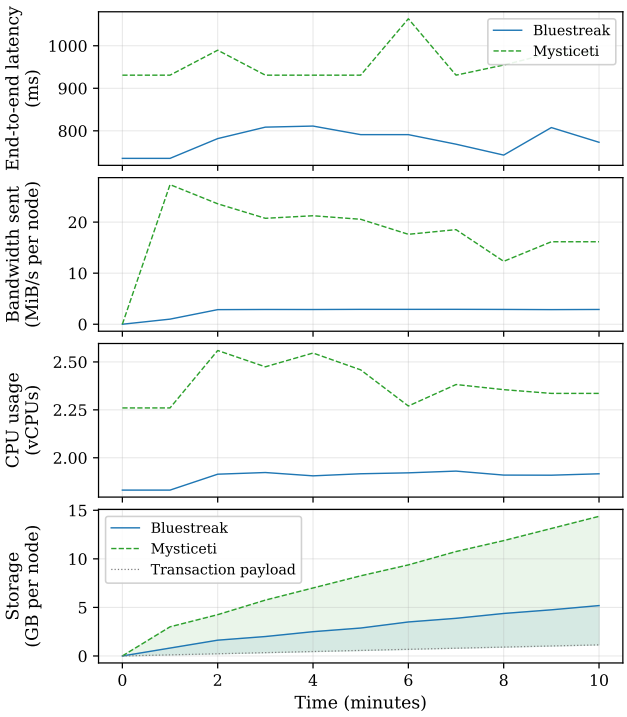

Figure 9. Stability with WAL-based storage (Tidehunter) at $n = 400$, load 4 000 tx/s on c5d.xlarge (4 vCPUs, 8 GiB RAM). **Top to bottom:** median latency, per-node bandwidth, per-node CPU, and per-node storage footprint.

storage. Bluestreak maintains ≈ 780 ms latency versus Mysticeti's ≈ 950 ms—notably, Mysticeti improves from 1.5 s under RocksDB (Figure 1) to sub-second with Tidehunter, confirming that LSM-tree compaction was a significant bottleneck at this committee size. Per-node bandwidth differs by ≈ 5× (2.9 vs. 15 MiB/s), and storage grows at ≈ 3× the rate (extrapolated to ≈ 5 GB vs. ≈ 15 GB per node after 10 minutes). At n=400 Mysticeti's per-block metadata reaches ≈ 13 KB, whereas Bluestreak's non-leader blocks carry ≈ 320 bytes—a 41× reduction. The storage gap is dominated by this metadata difference: with Mysticeti producing ≈ 1 800 blocks/s and Bluestreak ≈ 2 700 blocks/s, Mysticeti accumulates ≈ 13.3 GB of metadata in 10 minutes versus Bluestreak's ≈ 0.5 GB, closely matching the observed 3× storage ratio. These results confirm that Bluestreak's $O(1)$ block metadata translates into concrete throughput, bandwidth, and storage benefits once the storage-IO bottleneck is removed.

9 Discussion and Limitations

Bluestreak reduces per-block consensus metadata by making ordinary blocks sparse and concentrating richer structure into one leader block per round. This reallocation improves committee scaling in our evaluation (Figure 1), but it does not eliminate committee-scale work: the leader block still grows with n and may become the dominant bottleneck at much larger committees. At large committee sizes we also observe that adding a single random reference to a previous-round block in each non-leader block can significantly improve latency, because some per-node chains may not propagate to leaders in time; a random cross-reference provides an alternative path for inclusion in the leader's causal history. Exploring the optimal number of such additional references is left as future work.

Our empirical comparison has an additional limitation. To avoid confounding factors, we evaluate all protocols under the same implementation framework and with the same high-level parameter choices (e.g., leader timeouts and pacemaker settings). In practice, different designs may perform best under different storage engines, cryptographic implementations, batching strategies, timeout choices, and deployment scenarios, and protocol-specific tuning can materially change absolute numbers. We therefore provide the shared Rust benchmarking codebase as a platform to plug in optimized implementations and alternative components (e.g., WAL-based storage such as Tidehunter [7]) and to explore parameter sensitivity in a controlled setting. This can help distinguish intrinsic protocol costs (e.g., metadata scaling) from engineering bottlenecks and clarify the practical limits of DAG-based consensus.

10 Conclusion

Dense round-based DAG BFT protocols often couple committee scaling to per-block metadata, turning consensus blocks into a bandwidth and CPU bottleneck as n grows. Bluestreak rebalances this cost: metadata in ordinary blocks remains constant-size in n and only the leader block carries committee-scale ancestry, together with a commit rule and pull-based pacemaker proved safe and live under partial synchrony. Empirically, this yields nearly constant average metadata per block up to 400 validators and improves committee scaling, and with WAL-based storage it enables substantially higher peak throughput (up to ≈ 420k tx/s at $n = 120$) once storage bottlenecks are removed.

References

[1] Michael Anoprenko, Andrei Tonkikh, Alexander Spiegelman, Petr Kuznetsov, Anatoliy Zinovyev, and Konstantin Shprenger. 2025. DAGs for the Masses. https://doi.org/10.48550/arXiv.2506.13998 arXiv:2506.13998 [cs.DC]

[2] Balaji Arun, Zekun Li, Florian Suri-Payer, Sourav Das, and Alexander Spiegelman. 2025. Shoal++: High Throughput DAG BFT Can Be Fast and Robust!. In *USENIX NSDI '25*.

[3] Kushal Babel, Andrey Chursin, George Danezis, Anastasios Kichidis, Lefteris Kokoris-Kogias, Arun Koshy, Alberto Sonnino, and Mingwei Tian. 2025. Mysticeti: Reaching the Latency Limits with Uncertified DAGs. In *Network and Distributed System Security Symposium (NDSS)*. 1–18.

[4] Leemon Baird. 2016. The Swirlds Hashgraph Consensus Algorithm: Fair, Fast, Byzantine Fault Tolerance. Swirlds Tech Report SWIRLDS-TR-2016-01; revision dated March 18, 2018.

[5] Alexandra Boldyreva. 2003. Threshold signatures, multisignatures and blind signatures based on the gap-Diffie-Hellman-group signature scheme. In *International Workshop on Public Key Cryptography*. Springer, 31–46.

[6] Dan Boneh, Ben Lynn, and Hovav Shacham. 2001. Short signatures from the Weil pairing. In *International Conference on the Theory and Application of Cryptology and Information Security*. Springer, 514–532.

[7] Andrey Chursin, Lefteris Kokoris-Kogias, Alex Orlov, Alberto Sonnino, and Igor Zablotchi. 2026. Tidehunter: Large-Value Storage With Minimal Data Relocation. https://doi.org/10.48550/arXiv.2602.01873 arXiv:2602.01873 [cs.DB]

[8] Shir Cohen, Rati Gelashvili, Lefteris Kokoris-Kogias, Zekun Li, Dahlia Malkhi, Alberto Sonnino, and Alexander Spiegelman. 2022. Be Aware of Your Leaders. In *Financial Cryptography and Data Security (FC)*.

[9] George Danezis, Lefteris Kokoris-Kogias, Alberto Sonnino, and Alexander Spiegelman. 2022. Narwhal and Tusk: A DAG-based Mempool and Efficient BFT Consensus. In *Proceedings of the 17th European Conference on Computer Systems*. 34–50. https://doi.org/10.1145/3492321.3519594

[10] Cynthia Dwork, Nancy Lynch, and Larry Stockmeyer. 1988. Consensus in the Presence of Partial Synchrony. *J. ACM* 35, 2 (1988), 288–323. https://doi.org/10.1145/42282.42283

[11] Espresso Systems. 2025. Timeboost. https://github.com/EspressoSystems/timeboost GitHub repository, accessed April 2, 2026.

[12] Neil Giridharan, Florian Suri-Payer, Ittai Abraham, Lorenzo Alvisi, and Natacha Crooks. 2024. Autobahn: Seamless high speed BFT. In *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles*. 1–23. https://doi.org/10.1145/3694715.3695942

[13] IOTA Foundation. 2026. Consensus on IOTA. IOTA Documentation. https://docs.iota.org/about-iota/iota-architecture/consensus Accessed April 2, 2026.

[14] Idit Keidar, Oded Naor, O. Poupko, and Ehud Shapiro. 2023. Cordial Miners: Fast and Efficient Consensus for Every Eventuality. In *37th International Symposium on Distributed Computing (DISC 2023)*. https://doi.org/10.4230/LIPIcs.DISC.2023.26

[15] Tasos Kichidis, Lefteris Kokoris-Kogias, Arun Koshy, and Maxwell Tian. 2025. Mysticeti v2: Faster and Lighter Sui Transaction Processing. The Sui Blog. https://blog.sui.io/mysticeti-v2-sui-consensus/ Published November 6, 2025.

[16] Andrew Lewis-Pye and Ehud Shapiro. 2025. Morpheus Consensus: Excelling on trails and autobahns. https://doi.org/10.48550/arXiv.2502.08465 arXiv:2502.08465

[17] Nikita Polyanskii, Sebastian Müller, and Ilya Vorobyev. 2025. Starfish: A high throughput BFT protocol on uncertified DAG with linear amortized communication complexity. Cryptology ePrint Archive, Paper 2025/567. https://eprint.iacr.org/2025/567

[18] Serguei Popov. 2018. The Tangle. White paper, version 1.4.2, February 9, 2018.

[19] Serguei Popov, William Sanders, Hans Moog, Sebastian Mueller, Angelo Capossele, Andrii Kovalchuk, Alon Gal, Alisa Dmitrieva, William Buchanan, and Massimo Marchiori. 2020. The Coordicide. https://files.iota.org/papers/20200120_Coordicide_WP.pdf IOTA Foundation white paper, January 20, 2020.

[20] Mayank Raikwar, Nikita Polianskii, and Sebastian Müller. 2024. SoK: DAG-Based Consensus Protocols. In *Proceedings of the 6th IEEE International Conference on Blockchain and Cryptocurrency (ICBC)*. IEEE, Piscataway, NJ, USA, 1–18. https://doi.org/10.1109/ICBC59979.2024.10634358

[21] Omer Sadika, David Lachmish, Yehonatan Cohen Scaly, and Sheran Hussain. 2024. Ika: Fast, Decentralized and Secure Interoperability Powered by 2PC-MPC. https://ika.xyz/ika-whitepaper.pdf Version 1.0, October 30, 2024.

[22] Nibesh Shrestha, Rohan Shrothrium, Aniket Kate, and Kartik Nayak. 2025. Sailfish: Towards Improving the Latency of DAG-based BFT. In *2025 IEEE Symposium on Security and Privacy (SP)*.

[23] Nibesh Shrestha, Qianyu Yu, Aniket Kate, Giuliano Losa, Kartik Nayak, and Xuechao Wang. 2025. Optimistic, Signature-Free Reliable Broadcast and Its Applications. In *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security*. 3780–3794. https://doi.org/10.1145/3719027.3765220

[24] Alexander Spiegelman, Neil Giridharan, Alberto Sonnino, and Lefteris Kokoris-Kogias. 2022. Bullshark: DAG BFT Protocols Made Practical. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*. 2705–2718. https://doi.org/10.1145/3548606.3559361

[25] Giorgos Tsimos, Anastasios Kichidis, Alberto Sonnino, and Lefteris Kokoris-Kogias. 2024. Hammerhead: Leader Reputation for Dynamic Scheduling. In *IEEE International Conference on Distributed Computing Systems (ICDCS)*.

[26] Qianyu Yu, Giuliano Losa, Nibesh Shrestha, and Xuechao Wang. 2025. Angelfish: Leader, DAG, or Anywhere in Between. https://doi.org/10.48550/arXiv.2509.15847 arXiv:2509.15847

A Additional Implementation Details

Networking. Every node maintains a persistent TCP connection to every other node, forming a full mesh of $n(n-1)/2$ bidirectional links. Messages are length-prefixed and serialized with bincode. All four protocols use pull-based dissemination: each node broadcasts its own blocks to all peers and requests missing ancestors on demand.

Cryptography. Block are signed with Ed25519 (64-byte signatures) and digested with BLAKE3 ($\kappa = 256$ bits).

A block reference is a triple of round number (4 bytes), author index (2 bytes), and BLAKE3 digest (32 bytes), totalling 38 bytes. Mysticeti-BLS additionally uses BLS12-381 via the `blst` library, with 48-byte aggregate signatures and 96-byte public keys.

Storage. The default storage backend persists blocks in RocksDB with separate column families for block metadata, transaction payloads, and commit metadata. The store uses a 2 GB write buffer, pipelined writes, LZ4 upper-level compression, and Zstd dictionary compression on the bottommost level. For the WAL-based experiments in Q5, we use Tidehunter,[5] an append-only write-ahead-log engine that bypasses the LSM-tree entirely. Both backends are tuned for maximum write throughput on NVMe storage.

Concurrency. The runtime is built on Tokio. Each node runs one asynchronous task per peer connection, plus dedicated tasks for the consensus loop, leader timeout, and missing-block recovery.

Sailfish++. Our Sailfish++ implementation follows the certified consensus design of [23], in which every block is certified through a three-phase *signature-free* reliable broadcast (echo, vote, ready). To keep bandwidth low, the broadcast carries only the block reference (digest), not the full block with payload. The optimistic fast path delivers a block when $\lceil 5n/6 \rceil$ echo messages are collected; the slow path requires $2f+1$ ready messages. When a node cannot reference the previous-round leader, its block embeds either a *timeout certificate* ($\geq 2f+1$ signed timeout messages for that round) or a *no-vote certificate* ($\geq 2f+1$ signed no-votes for the leader slot), proving to receivers that the leader was absent. The commit rule operates on all received blocks, including uncertified ones: a leader certified at round r commits once $2f+1$ round-$(r+1)$ blocks reference it.

Mysticeti-BLS. Mysticeti-BLS is an instance of the Mysticeti-L variant from [17]. In plain Mysticeti, leader certification is proved by the causal history: a leader is certified when $2f+1$ blocks in its causal future reference it, requiring $O(n)$ references per block. In Bluestreak, certificate claims are explicit flags in block metadata but still require the verifier to inspect locally stored blocks. Mysticeti-BLS takes a third approach: each node votes for the leader by including a BLS12-381 partial signature in its block. Once $2f+1$ partial signatures are collected, the node aggregates them into a single compact certificate (48-byte aggregate signature plus a bitmask of signers) and embeds it in a subsequent block. The aggregate signature is a self-contained, explicit proof of certification—no causal-history traversal or local block verification is needed. Two certificate types are maintained: *round certificates*—when a node advances to round r it broadcasts a BLS partial signature on the value r, and once $2f+1$ partials are collected they are aggregated into a round certificate—and *leader certificates* (quorum votes for a specific leader block). BLS verification is offloaded to a pool of dedicated worker threads; consensus-critical signatures (round and leader) are verified with priority, while data-availability signatures are deferred under backlog to avoid stalling the commit pipeline. The resulting per-block overhead is ≈ 730 bytes regardless of committee size.

RTT latency matrix. Table 3 shows the round-trip times (in milliseconds) between the ten AWS regions used by the synthetic latency model. The one-way delay applied to each node pair is half the table entry, plus ±3% uniform jitter.

Table 3. Round-trip times (ms) between virtual regions.

	us-east-1	*us-west-1*	*ca-central-1*	*eu-west-1*	*eu-south-1*	*eu-north-1*	*sa-east-1*	*ap-south-1*	*ap-se-1*	*ap-ne-1*
us-east-1	1	14	104	112	198	65	68	110	201	146
us-west-1	14	1	106	122	196	78	67	103	189	142
ca-central-1	104	106	1	215	281	163	29	50	143	238
eu-west-1	112	122	215	1	309	175	176	220	299	254
eu-south-1	198	196	281	309	1	137	254	268	150	101
eu-north-1	65	78	163	175	137	1	127	172	226	108
sa-east-1	68	67	29	176	254	127	1	38	125	199
ap-south-1	110	103	50	220	268	172	38	1	148	245
ap-se-1	201	189	143	299	150	226	125	148	1	140
ap-ne-1	146	142	238	254	101	108	199	245	140	1

B Full Safety Proof

Lemma B.1 (At most one certified block per slot)**.** *At most one leader block from a given leader slot can be certified (i.e., receive $2f+1$ votes at round $r+1$).*

Proof. For contradiction, assume two distinct blocks L and L' from leader slot (p_i, r) each receive votes from $2f+1$ distinct nodes at round $r+1$. Since $n = 3f+1$, the intersection of these two sets of voters contains at least $(2f+1)+(2f+1)-(3f+1) = f+1$ nodes. Each node in the intersection voted for both L and L' at round $r+1$. Since honest nodes create exactly one block per round with at most one leader reference, they vote for at most one leader block. Therefore, all $f+1$ nodes in the intersection are Byzantine, contradicting the assumption that at most f nodes are Byzantine. □

[5] https://github.com/MystenLabs/tidehunter

Lemma B.2 (Directly committed leader blocks are certified)**.** *If an honest node directly commits leader block L for slot (p_i, r), then L is certified.*

Proof. If the node directly commits L, then its local DAG contains $2f+1$ round-$(r+2)$ blocks from distinct nodes that make a certificate claim for L. Among these $2f+1$ creators, at most f are Byzantine, so at least $f+1$ are honest. Each such honest creator makes the claim only after genuinely observing that $2f+1$ round-$(r+1)$ blocks vote for L. Hence L is certified. □

Lemma B.3 (Certified anchors preserve certification)**.** *Suppose an honest node indirectly commits leader block L in leader slot (p_i, r) using anchor block A. If A is certified, then L is certified.*

Proof. Since the node commits L indirectly using A, by the indirect commit rule $\mathsf{hist}(A)$ contains a certificate claim for L in slot (p_i, r). Since A is certified, there are $2f+1$ round-$(r'+1)$ voting blocks for A, where r' is the round of A. Among their creators, at most f are Byzantine, so at least $f+1$ are honest. Pick one such honest creator. Its voting block references A, so A is referenceable in that honest creator's local DAG at the moment that voting block is created. By the definition of referenceability, every certificate claim in $\mathsf{hist}(A)$ is locally provable in that honest creator's local DAG. In particular, the certificate claim for L in slot (p_i, r) is locally provable in that honest creator's local DAG. By the definition of local provability, that local DAG contains $2f+1$ round-$(r+1)$ voting blocks from distinct nodes for L. Hence L is certified. □

Corollary B.4 (Committed leader blocks are certified)**.** *If an honest node commits leader block L for slot (p_i, r), then L is certified.*

This follows from Lemma B.2 and repeated application of Lemma B.3 along the finite anchor chain of an indirectly committed block.

Corollary B.5 (Same commit)**.** *No two honest nodes commit distinct leader blocks for the same leader slot.*

This follows immediately from Lemma B.1 and Corollary B.4.

Lemma B.6 (Certified proposals cannot be directly skipped)**.** *If leader block L from slot (p_i, r) is certified, then no honest node directly skips proposal L.*

Proof. Let V be a set of $2f+1$ nodes whose round-$(r+1)$ blocks vote for L. Suppose, for contradiction, that some honest node directly skips proposal L. Then its local DAG contains a set N of $2f+1$ round-$(r+1)$ blocks from distinct nodes that are non-votes for L. Since $n = 3f+1$,

$$|V \cap N| \geq |V| + |N| - n \geq (2f+1) + (2f+1) - (3f+1) = f+1.$$

Each node in $V \cap N$ both votes for L and produces a non-vote for L at round $r+1$. An honest node creates only one block per round, so this is impossible for honest nodes. Therefore all $f+1$ nodes in $V \cap N$ are Byzantine, contradicting the bound of at most f Byzantine nodes. □

The following corollary is immediate from Lemma B.6 and the definition of the slot-level direct-skip rule.

Corollary B.7 (Certified slots cannot be directly skipped)**.** *If some leader block in slot (p_i, r) is certified, then no honest node directly skips that slot.*

Lemma B.8 (Directly committed slot cannot be indirectly skipped)**.** *If an honest node directly commits a leader slot (p_i, r), then no honest node will indirectly skip it.*

Proof. Suppose an honest node directly commits leader block L_r in slot (p_i, r). Then there is a set C of at least $2f+1$ nodes whose round-$(r+2)$ blocks make a certificate claim for L_r. Consider any honest node that later applies the indirect rule to slot (p_i, r), and let A be the anchor block it uses; let $r' \geq r+3$ denote the round of A.

Let R be the set of nodes whose round-$(r'-1)$ blocks are referenced by A, so $|R| \geq 2f+1$. Since $n = 3f+1$,

$$|R \cap C| \geq |R| + |C| - n \geq (2f+1) + (2f+1) - (3f+1) = f+1.$$

At most f nodes are Byzantine, so $R \cap C$ contains at least one honest node. Pick an honest node in $R \cap C$. Its round-$(r'-1)$ block is referenced by A, so following that node's self-chain downward shows that its round-$(r+2)$ block B belongs to $\mathsf{hist}(A)$. Since the node lies in C, block B makes a certificate claim for L_r. Hence $\mathsf{hist}(A)$ contains a certificate claim for L_r, so the indirect rule commits slot (p_i, r) rather than skipping it. Since the choice of the honest node was arbitrary, no honest node will indirectly skip slot (p_i, r). □

Lemma B.9 (Consistent states)**.** *All honest nodes have a consistent state for each leader slot: if two honest nodes have both decided the state of a slot, then both either commit the same block or both skip.*

Proof. For contradiction, assume that honest nodes p_1 and p_2 have inconsistent states for some leader slot. Pick the slot from the latest round r where one node commits and the other skips. Without loss of generality, assume p_1 commits the leader block and p_2 skips the leader slot of round r.

By Lemma B.4, the leader block committed by p_1 is certified. Therefore, by Corollary B.7, node p_2 cannot skip slot (p_i, r) directly. So p_2 reaches its skip decision indirectly.

We consider two cases for p_1:

Case 1: p_1 committed directly. Then p_2 indirectly skips a directly committed slot, contradicting Lemma B.8.

Case 2: p_1 committed indirectly. Both nodes decided indirectly. Let r_1 and r_2 be the anchor rounds used by p_1 and p_2, respectively. By the indirect rule, for each $j \in \{1, 2\}$ node p_j marks every leader slot in rounds $[r+3, r_j)$ as SKIP and the anchor slot at round r_j as COMMIT.

We must have $r_1 = r_2$; otherwise, without loss of generality assume $r_1 < r_2$. Then p_2 marks slot (p_i, r_1) as SKIP (since

$r_1 \in [r+3, r_2)$) while p_1 marks it as COMMIT (since r_1 is its anchor), contradicting the choice of r as the latest round where one honest node commits and another skips.

By Corollary B.5, both nodes use the same committed anchor block A at anchor round $r_1 = r_2$. The indirect decision for slot (p_i, r) depends only on the causal history $\mathsf{hist}(A)$ of this anchor block. Since that history is the same for both nodes, both nodes must reach the same indirect decision for round r. This contradicts the assumption that p_1 commits and p_2 skips. □

Corollary B.10 (Same sequence of committed leaders)**.** *For any two honest nodes, the sequence of committed leader blocks computed by one node is a prefix of the sequence computed by the other node.*

This is immediate from Lemma B.9: if the sequences differed at their first common slot, then the two nodes would have inconsistent states for that slot.

Lemma B.11 (Unique payload per author-round pair)**.** *If two honest nodes append payloads corresponding to author-round pair (p_k, ρ), then they append the same payload.*

Proof. Fix two honest nodes p_i and p_j and assume each appends a payload for the author-round pair (p_k, ρ). By the output rule in the previous section, there exist blocks B and B' with $\mathsf{round}(B) = \mathsf{round}(B') = \rho$ and $\mathsf{author}(B) = \mathsf{author}(B') = k$ such that p_i appends $\mathsf{payload}(B)$ and p_j appends $\mathsf{payload}(B')$, and each node appends the payload for the pair (k, ρ) when processing the first committed leader whose history contains any block from that pair.

By Corollary B.10, one node's committed-leader sequence is a prefix of the other's, so this first committed leader is the same at both nodes; call it L. Since L is in both nodes' local DAGs and blocks are added only together with their referenced history, $\mathsf{hist}(L)$ is uniquely determined by hash references and is the same at both nodes. Both nodes therefore compute the same list $\mathsf{sort}(\mathsf{hist}(L))$. When processing $\mathsf{sort}(\mathsf{hist}(L))$, the first block with $(\mathsf{author}, \mathsf{round}) = (k, \rho)$ is the same at both nodes, and later blocks with the same pair are ignored. Hence $B = B'$ and therefore the appended payloads are identical. □

Lemma B.12 (Consistent output order)**.** *If two honest nodes append the payloads of blocks B and B', then they append them in the same relative order. In particular, if two honest nodes both append the payload of a block B, then both append it at the same position in the output sequence.*

Proof. By Corollary B.10, for any two honest nodes, one node's committed-leader sequence is a prefix of the other's. For each committed leader L, the causal history $\mathsf{hist}(L)$ is uniquely determined by hash references and therefore is the same at all nodes. By the payload-ordering rule above, each node processes committed leaders in increasing slot order and, for each committed leader L, appends the payloads of the blocks that become newly reachable at L (in the order sort), appending at most one payload for each pair (author, round). The procedure is deterministic and only appends new payloads when the committed-leader sequence grows, so one node's output sequence is a prefix of the other's. Therefore any two honest nodes that both append the payloads of B and B' append them in the same relative order. □

Theorem B.13 (Safety)**.** *For any two honest nodes, their output sequences are consistent: one is a prefix of the other. In particular, if both nodes append a payload for the same author-round pair, then the payloads are identical.*

Proof. Prefix consistency follows from Lemma B.12. The per-author-round uniqueness claim follows from Lemma B.11. □

C Full Liveness Proof

In this section we prove that after GST the output sequence keeps growing at every honest node. The first step is a pair of post-GST synchronization lemmas for round advancement and block creation. Later lemmas combine these with leader-slot decision progress to show that every payload proposed by an honest node is eventually appended everywhere.

Let $r_{\max}$ be the largest round reached by an honest node at the moment of GST.

Lemma C.1 (Round synchronicity after GST)**.** *Fix a round $r > r_{\max}$. All honest nodes enter round r within a time interval of length 4Δ.*

Proof. Fix any round r higher than any round reached by an honest node at the moment of GST. Let p_1 be the first honest node that enters round r, and let t_1 be this entry time. Since rounds only increase, we have $t_1 \geq \mathsf{GST}$ and post-GST message delays are bounded by Δ.

By condition B1, upon entering round r node p_1 immediately broadcasts $(\mathsf{advance}, r)$. Consider any other honest node p. By time $t_1 + \Delta$, node p receives $(\mathsf{advance}, r)$. If it has not yet entered round r upon receiving this message, it starts an advance timer of length δ_A (P1). If p is still below round r when the timer expires, it sends a pull request to p_1 for the data needed to advance up to round r (P2). The pull request and the response are delivered within two further message delays 2Δ, so p receives the response by time $t_1 + \Delta + \delta_A + 2\Delta$.

The response contains, for each missing round $k < r$, a set S_k of $2f+1$ referenceable round-k blocks from distinct nodes (enough to satisfy A1), together with all missing referenced history (transitively) and voting blocks needed to add the blocks in S_k to p's local DAG and make them referenceable. This allows p to catch up without waiting for timeouts: for each round k it is missing, once it adds the blocks in S_k to its local DAG it satisfies C3 and can create its own round-k block, and then advances using A1 and A2. Since local computation is negligible relative to message delay, p enters round r by time $t_1 + 3\Delta + \delta_A$.

Since we set $\delta_A = \Delta$ in this section, p enters round r by time $t_1 + 4\Delta$. Since p was arbitrary, all honest nodes enter round r by time $t_1 + 4\Delta$, proving that their round-r entry times lie in an interval of length 4Δ. □

Lemma C.2 (Block synchronicity after GST)**.** *Fix a round $r > r_{\max}$. All honest nodes create their round-r blocks within a time interval of length 4Δ.*

Proof. Fix a round $r > r_{\max}$. Let p_1 be the first honest node that creates a round-r block, and let t_1 be this creation time. This creation cannot be triggered by condition C3, since before any honest round-r block is created there can exist round-r blocks from at most f distinct nodes (the Byzantine nodes), which is insufficient for the $2f + 1$ distinct-nodes threshold in C3.

By Lemma C.1, all honest nodes enter round r within a time interval of length 4Δ.

We now show that all honest nodes also create their round-r blocks within time 4Δ of t_1. Fix any honest node p.

If p_1 creates its round-r block due to condition C2, then t_1 is the expiration time of p_1's round-r timeout. All honest nodes start their round-r timeout upon entering round r, so their timeout expirations differ by at most the spread of round-entry times, which is at most 4Δ. Therefore p creates its round-r block by time $t_1 + 4\Delta$.

Otherwise, p_1 creates its round-r block due to condition C1. By condition B2, node p_1 immediately broadcasts its round-r block, so every honest node receives it by time $t_1 + \Delta$. If node p is missing any prerequisites needed before adding this block to its local DAG (e.g., referenced history blocks) or to validate certificate claims appearing in its causal history (i.e., the voting blocks needed to make those claims locally provable and thus make the block referenceable), then by the delayed-pull recovery rules it starts a pull-delay timer upon receipt (P1) and, if still missing data when the timer expires, pulls the missing prerequisites (P2). Since $\delta_D = \Delta$ and post-GST message delays are bounded by Δ, node p receives the pulled prerequisites within 4Δ of time t_1. By time $t_1 + 4\Delta$, node p has entered round r and has the information needed to satisfy C1, and thus it creates its own round-r block by then.

Since p was arbitrary, every honest node creates its round-r block by time $t_1 + 4\Delta$. Therefore all honest nodes create their round-r blocks within a time interval of length 4Δ. □

Lemma C.3 (Timely referenceability before timeout)**.** *Fix a round $r > r_{\max}$. All blocks created by honest nodes in round $r - 1$ are referenceable at every honest node before that node's round-r timeout expires.*

Proof. Let t_{GST} be the time of GST, and let t_{last} be the creation time of the last honestly created block of round $r - 1$. After GST, messages sent by honest nodes are delivered within Δ. Therefore by time

$$t_{\text{recv}} = \max(t_{\text{GST}}, t_{\text{last}}) + \Delta$$

every honest node has received all honestly created blocks of round $r - 1$.

Fix any honest node p and any honestly created round-$(r-1)$ block B. Let t_B be the time when p receives B. If B is not yet referenceable at p upon receipt (e.g., because p is missing referenced history blocks or the voting blocks needed to justify inherited certificate claims), then by the delayed pull recovery rules p starts a pull-delay timer of length $\delta_D = \Delta$ upon receipt (P1) and, if still missing prerequisites when the timer expires, pulls them from the sender of B (P2). Post-GST message delays are bounded by Δ, so the pull request and response are delivered within two further message delays 2Δ. Therefore p receives all pulled prerequisites by time $t_B + \delta_D + 2\Delta = t_B + 3\Delta$. By the pull-response definition, this response is self-contained: it includes all missing referenced history blocks (transitively) and all missing voting blocks needed to make inherited certificate claims locally provable. Thus B becomes referenceable at p by time $t_B + 3\Delta$.

Since B and p were arbitrary, by time $t_{\text{recv}} + 3\Delta$ every honest node has every honestly created round-$(r - 1)$ block referenceable in its local DAG.

Let p_1 be the first honest node that enters round r, and let t_1 be its entry time. By Lemma C.1, all honest nodes enter round r by time $t_1 + 4\Delta$. Every honest node creates its own round-$(r - 1)$ block before entering round r (A2), so we have $t_{\text{last}} \leq t_1 + 4\Delta$. Since $r > r_{\max}$, this happens after GST, so $t_1 \geq t_{\text{GST}}$ and therefore $t_{\text{recv}} \leq t_1 + 5\Delta$. Thus $t_{\text{recv}} + 3\Delta \leq t_1 + 8\Delta$.

All honest nodes start their round-r timeout upon entering round r, so the earliest timeout expiration is at time $t_1 + \delta_{LT} = t_1 + 8\Delta$. Therefore every honestly created round-$(r-1)$ block is referenceable at every honest node before that node's round-r timeout expires. □

Lemma C.4 (Consistent leader reference)**.** *Suppose that for some round r every honest node has, before its round-r timeout expires, a referenceable leader block L for round $r - 1$ in its local DAG. Then every honest round-r block votes for L.*

Proof. Fix an honest node p and consider how it creates its round-r block.

If block creation is triggered by condition C1, then by L1 the local DAG of p contains the round-$(r - 1)$ leader block, and L1 requires this leader block to be referenceable at p when p creates its round-r block. Thus, when constructing its block, p includes a leader reference to L and therefore votes for L.

If block creation is triggered by condition C2, then by assumption the leader block L is already referenceable at p before the timeout expires. Since p creates its round-r block exactly when its round-r timeout expires and DAG_p only grows over time, L is referenceable when p creates its round-r block. Therefore p includes a leader reference to L and votes for it.

Finally, consider the case where block creation is triggered by condition C3. Let W be the set of the first $f+1$ honest nodes to create a round-r block. For every node in W, block creation cannot be triggered by C3: before the k-th honestly created round-r block (for any $k \le f+1$), there exist round-r blocks from at most $f+(k-1) \le 2f$ distinct nodes (at most f Byzantine nodes and $k-1$ honest nodes), which is insufficient for the $2f+1$ distinct-nodes threshold in C3. Therefore every node in W creates its round-r block due to C1 or C2, and thus votes for L by the arguments above.

Now consider any honest node p whose block creation is triggered by C3. At that moment, p's local DAG contains $2f+1$ *referenceable* round-r blocks from distinct nodes. At most f of them can be Byzantine, so at least $f+1$ of them were created by honest nodes. Since there are only f honest nodes outside W, among these $f+1$ honest blocks there exists at least one block B created by a node in W. As shown above, B votes for L, so $L \in \mathsf{hist}(B)$.

Since B is referenceable at p and $\mathsf{hist}(L) \subseteq \mathsf{hist}(B)$, every certificate claim appearing in $\mathsf{hist}(L)$ is locally provable at p as well. Therefore L is referenceable at p when p creates its round-r block, and so p also votes for L. □

Lemma C.5 (Honest leaders commit)**.** *Fix a round $r > r_{\max}$ and suppose the leader of round r is honest and creates a leader block L. Then every honest node eventually commits L.*

Proof. By Lemma C.3 applied to round $r+1$, leader block L is referenceable at every honest node before its round-$(r+1)$ timeout expires. Therefore, by Lemma C.4 (applied to round $r+1$ and leader block L), every honest round-$(r+1)$ block votes for L. Since there are at least $n-f \ge 2f+1$ honest nodes, the certificate pattern holds for L: at least $2f+1$ round-$(r+1)$ blocks from distinct nodes vote for L.

Now fix any honest node p. Consider how p creates its round-$(r+2)$ block.

If block creation is triggered by C1, then L2 holds. Since every honest round-$(r+1)$ block votes for L, there exist at most f round-$(r+1)$ non-votes for L, which is insufficient to directly skip L. Therefore L2 must hold via the vote option, meaning that at block-creation time the local DAG of p contains $2f+1$ *referenceable* round-$(r+1)$ votes for L. Thus p's round-$(r+2)$ block makes a certificate claim for L (explicitly in `certClaim` for non-leaders, and implicitly via the ancestor list for leaders).

If block creation is triggered by C2, then p creates its round-$(r+2)$ block when its round-$(r+2)$ timeout expires. By Lemma C.3 applied to round $r+2$, all honestly created round-$(r+1)$ blocks are referenceable at p before that timeout expires. In particular, by timeout expiry p has $2f+1$ round-$(r+1)$ votes for L in its local DAG, and so p's round-$(r+2)$ block makes a certificate claim for L.

Finally, suppose block creation is triggered by C3. Let W be the set of the first $f+1$ honest nodes to create a round-$(r+2)$ block. As in Lemma C.4, no node in W can create its block via C3, so every node in W creates its round-$(r+2)$ block via C1 or C2 and therefore makes a certificate claim for L. When p creates its round-$(r+2)$ block, by C3 its local DAG contains $2f+1$ *referenceable* round-$(r+2)$ blocks from distinct nodes. At most f of them can be Byzantine, so at least $f+1$ of them were created by honest nodes; since there are only f honest nodes outside W, at least one of these blocks, call it B, was created by a node in W. Since B is referenceable at p and makes a certificate claim for L, this claim is locally provable at p, meaning that p's local DAG contains $2f+1$ round-$(r+1)$ votes for L. Therefore p's round-$(r+2)$ block makes a certificate claim for L.

Thus every honest node creates a round-$(r+2)$ block that makes a certificate claim for L. By Lemma C.3 applied to round $r+3$, all honestly created round-$(r+2)$ blocks are referenceable at every honest node before its round-$(r+3)$ timeout expires. In particular, every honest node's local DAG eventually contains $2f+1$ round-$(r+2)$ blocks from distinct nodes that make a certificate claim for L. Therefore the direct commit rule for L eventually becomes applicable at every honest node; if an honest node has not already committed L earlier (e.g., by the indirect rule), it commits L at that point. □

Lemma C.6 (Three consecutive honest leaders)**.** *In the round-robin leader schedule, in any $n+2 = 3f+3$ consecutive rounds there exist three consecutive rounds whose leaders are honest.*

Proof. Consider any window of $n+2$ consecutive rounds $r_0, r_0+1, \ldots, r_0+n+1$. Since the leader schedule is round-robin, the sequence of leaders in this window has the form

$$p_s,\ p_{s+1},\ \ldots,\ p_{s+n-1},\ p_s,\ p_{s+1},$$

for some index s, where indices are taken modulo n. The window contains n groups of three consecutive rounds, namely the groups starting at rounds $r_0, r_0+1, \ldots, r_0+n-1$. The corresponding leader triples are

$$(p_{s+t},\ p_{s+t+1},\ p_{s+t+2}) \qquad \text{for } t \in \{0, \ldots, n-1\},$$

with indices modulo n. In these n triples, each node appears exactly three times in total: once as the first leader of a triple, once as the second, and once as the third.

Since there are at least $2f+1$ honest nodes, the total number of honest-leader occurrences across the n triples is at least $3(2f+1) = 6f+3$. If every triple contained at most two honest leaders, then the total number of honest-leader occurrences would be at most $2n = 2(3f+1) = 6f+2$, a contradiction. Therefore some triple contains three honest leaders, i.e., there exist three consecutive rounds whose leaders are honest. □

Lemma C.7 (All leader slots are decided)**.** *After GST, every leader slot is eventually decided: at every honest node, its state eventually becomes either* COMMIT *or* SKIP.

Proof. Fix any leader slot in round r. Let $r_0 = \max(r, r_{\max})$. By Lemma C.6, among rounds $r_0, r_0+1, \ldots, r_0+n+1$ there

exist three consecutive rounds $k, k+1, k+2$ whose leaders are honest. By Lemma C.5, the leader blocks for rounds $k, k+1, k+2$ are eventually committed, and thus these three leader slots are marked COMMIT at every honest node.

Now consider any leader slot from a round $r' \leq k-1$ that remains UNDECIDED after applying the direct decision rules. We show that it is decided by the indirect decision rule. Consider an honest node and process leader slots in decreasing round order starting from $k-1$ down to r'. When applying the indirect decision rule to a slot in round j, all slots in rounds strictly higher than j have already been marked COMMIT or SKIP by this backward iteration. Therefore, when the rule scans leader slots starting from round $j+3$ and finds the first slot that is not SKIP, that slot is already marked COMMIT and provides a committed anchor. The indirect rule then decides the round-j slot by checking whether the anchor history contains a certificate claim for that slot: it marks COMMIT if the claim is present and SKIP otherwise.

Applying this argument iteratively, every slot in rounds up to $k-1$ is eventually marked either COMMIT or SKIP. Since r was arbitrary, every leader slot is eventually decided after GST. □

Lemma C.8 (Every honest block appears in a committed leader history)**.** *Let B be a block created by an honest node. Then there exists a leader block L such that $B \in hist(L)$ and every honest node eventually commits L.*

Proof. Let $q = \mathsf{author}(B)$ and $r_B = \mathsf{round}(B)$. Pick any honest node p_ℓ. Since the leader schedule is round-robin, p_ℓ is leader in infinitely many rounds. Choose one such leader round r with

$$r - n + 1 > \max(r_B, r_{\max}),$$

and let C be the round-$(r-n+1)$ block created by q. Because q is honest and $r - n + 1 > r_{\max}$, Lemma C.2 implies that C is created.

By the mandatory self-reference rule for non-leader blocks and leader blocks, following q's self-chain downward from C reaches every earlier block created by q, in particular B. Hence $B \in \mathsf{hist}(C)$.

Apply Lemma C.3 to round $r - n + 2$. It implies that C is referenceable at every honest node before that node's round-$(r - n + 2)$ timeout expires. Since local DAGs only grow, C remains referenceable at every honest node thereafter. In particular, C is referenceable in p_ℓ's local DAG when p_ℓ creates its leader block L in round r.

The previous leader round of p_ℓ is $r - n = (r - n + 1) - 1$. Therefore, before round r, node p_ℓ has never had a leader block in which it could have referenced any block of q from round $r - n + 1$ or later. By the honest leader-block construction rule, when p_ℓ creates L it includes for node q the latest referenceable block of q known locally at that time that has not already been referenced earlier. That referenced block has round at least $r - n + 1$, so its causal history contains C, and therefore also contains B. Thus $B \in \mathsf{hist}(L)$.

Since $r > r_{\max}$ and L is created by an honest leader, Lemma C.5 implies that every honest node eventually commits L. □

Theorem C.9 (Liveness)**.** *After GST, every payload proposed by an honest node is eventually appended to the output sequence of every honest node. Combined with Theorem B.13, the protocol implements a BFT state machine replication.*

Proof. Fix a block B created by an honest node. By Lemma C.8, there exists a leader block L such that $B \in \mathsf{hist}(L)$ and every honest node eventually commits L.

By Lemma C.7, every leader slot up to the round of L is eventually marked either COMMIT or SKIP at every honest node. Therefore every honest node eventually processes L in its committed leader sequence.

By the output sequencing rule, when processing L each honest node appends the payloads of the blocks in $\mathsf{hist}(L)$ that become newly reachable through L. Since $B \in \mathsf{hist}(L)$, the payload of B is appended no later than when L is processed; if it became newly reachable earlier through a previously committed leader, then it was appended even earlier. Thus the payload of B is eventually appended by every honest node, establishing liveness. □

D Happy-Case Latency and Bandwidth Analysis

This section explains the Bluestreak values reported in Table 1 of the main paper. The latency column in Table 1 is computed under the idealized assumptions below for all protocols in the table. We then derive the Bluestreak average end-to-end latency under those assumptions and the Bluestreak happy-case communication under the standard post-GST happy-case conditions.

Idealized latency conditions. For the latency comparison we use the following idealized assumptions:

- All nodes are honest.
- Every message is delivered after delay δ.
- Nodes enter each round simultaneously.
- In every round r, the leader references all blocks from round $r-1$.
- Transactions arrive continuously at a uniform rate and are buffered until the next local block creation.

These assumptions are intentionally optimistic, so the latency values in Table 1 should be read as lower-bound style comparison points rather than exact predictions. In reality, leaders may omit some round-$(r-1)$ blocks, nodes enter rounds at different times, and message delays need not be exactly δ; each of these effects can increase latency.

Average end-to-end latency. Under the assumptions above, rounds advance at cadence δ. A leader block created in round r is directly committed after the creation of round-$(r+2)$ blocks, so its consensus latency is 3δ. Because every round-r

leader references all blocks from round $r-1$, every non-leader block from round $r-1$ enters the ordered leader one round later and therefore has consensus latency 4δ. Each round contains one leader block and $n-1$ non-leader blocks, so the average consensus latency is

$$\frac{3\delta + 4\delta(n-1)}{n} = 4\delta - \frac{\delta}{n}.$$

Transactions wait on average another $\frac{1}{2}\delta$ before being included in the next created block. Therefore the average end-to-end latency is

$$4\delta - \frac{\delta}{n} + \frac{1}{2}\delta = 4.5\delta - \frac{\delta}{n},$$

which is reported in Table 1 as 4.5δ.

Happy-case bandwidth. We next justify the happy-case communication entry for Bluestreak in the main paper's comparison table under the standard happy-case conditions after GST. We measure bandwidth as the total number of bytes sent over the network in one round, counting a broadcast message once per recipient.

Bandwidth for block references. We first count the bandwidth spent solely on explicit block references that form DAG edges (i.e., the references in `self`, $\texttt{leader}_{r-1}$, and `ancestors`, excluding the certificate-claim hash field `certClaim`). Let ℓ_{ref} be the byte-length of a block reference. In one round there is one leader block, whose list `ancestors` contains at most one reference per node and therefore at most n references. Each of the remaining $n-1$ blocks contains the mandatory self reference `self` and the optional leader reference $\texttt{leader}_{r-1}$, and therefore at most 2 references. Thus, across the n blocks of a round, the total number of explicit references is at most $n + 2(n-1) = 3n-2$. Since each block is broadcast to all n recipients, the total bandwidth spent on these references in one round is at most $(3n-2)n\ell_{\text{ref}}$ bytes. If every non-leader block includes $\texttt{leader}_{r-1}$ and the leader includes a reference for every node in `ancestors`, then the number of references is exactly $3n-2$ and the reference bandwidth is exactly $(3n-2)n\ell_{\text{ref}}$ bytes.

Theorem D.1 (Happy-case bandwidth per round)**.** *Fix any round r after GST in an execution in which all nodes follow the protocol and all messages are delivered within delay Δ. Then the pull step (P2) is never executed in round r. Let M be the total payload size (in bytes) across all round-r blocks. Let ℓ_{sig} be the byte-length of a signature. Let $\ell_{\text{hash}} = \kappa/8$ be the byte-length of a hash digest. Let ℓ_{adv} and ℓ_{blk} be upper bounds on the byte-length of the constant-size unsigned contents of, respectively, a round-advance message $(\texttt{advance}, r)$ and a block header, excluding block references, certificate-claim hashes, and payload. Then the total bandwidth in round r is at most*

$$Mn + (3n-2)n\ell_{\text{ref}} + (n-1)n\ell_{\text{hash}} + 2n^2\ell_{\text{sig}} + n^2(\ell_{\text{adv}} + \ell_{\text{blk}}).$$

Concrete parameterization. For the concrete estimates used in practice, we take $\ell_{\text{sig}} = 64$ bytes for Ed25519 signatures, $\ell_{\text{hash}} = 32$ bytes for BLAKE3 digests, $\ell_{\text{ref}} = 38$ bytes for a block reference, $\ell_{\text{adv}} = 5$ bytes for an advance message (1 byte tag and 4 bytes for the round number), and $\ell_{\text{blk}} = 5$ bytes for the fixed unsigned block-message content (1 byte tag and 4 bytes for the round number). Substituting these values into the theorem gives the concrete bound

$$(M + 284n - 108)n$$

bytes per round.

Proof. We first show that the pull step (P2) is never executed. Consider any node p receiving either (i) a block B whose prerequisites are not yet satisfied, or (ii) a round-advance message $(\texttt{advance}, r)$ while still below round r. Let t be the receipt time of the triggering message, and let t_{send} be its send time. Under the theorem assumptions, nodes broadcast the blocks they create (B2), and every such broadcast is delivered within delay Δ. Moreover, by the local-DAG construction and the round-advance and block-creation conditions, a node sends a block or $(\texttt{advance}, r)$ only after obtaining the prerequisite blocks in its own local DAG, and thus only after those blocks have been created and broadcast. Therefore, any prerequisite block that has already been created and broadcast by time t_{send} is received by p by time $t_{\text{send}} + \Delta \le t + \Delta$. Since we set $\delta_D = \delta_A = \Delta$, any temporarily missing prerequisite arrives before the pull-delay timer expires, so p never reaches (P2).

We now count bandwidth in round r. Each node broadcasts one signed round-advance message $(\texttt{advance}, r)$ (B1). Each such message contains one signature and at most ℓ_{adv} bytes of unsigned content, so round-advance messages contribute at most

$$n^2(\ell_{\text{sig}} + \ell_{\text{adv}})$$

bytes.

Each node also broadcasts one round-r block (B2). Across the n blocks of a round, the total number of explicit block references that form DAG edges is at most $3n-2$, contributing $(3n-2)n\ell_{\text{ref}}$ bytes (as above). In addition, each of the $n-1$ non-leader blocks carries one certificate-claim hash `certClaim` of length ℓ_{hash} bytes, contributing a further $(n-1)n\ell_{\text{hash}}$ bytes. Therefore, reference-related metadata in one round contributes at most

$$(3n-2)n\ell_{\text{ref}} + (n-1)n\ell_{\text{hash}}$$

bytes.

Each round-r block also contains one signature and at most ℓ_{blk} bytes of constant-size unsigned header content, excluding block references, certificate-claim hashes, and payload. Thus, block headers contribute at most

$$n^2(\ell_{\text{sig}} + \ell_{\text{blk}})$$

bytes.

Finally, payload contributes Mn bytes, since every payload byte in a round-r block is delivered to all n recipients by broadcast. Summing all contributions yields the claimed bound. □

E Protocol Pseudocode

E.1 Sparse DAG Model Recap

The pseudocode uses the same block format and referenceability notion as the main paper. Non-leader blocks carry a mandatory self-reference, an optional previous-leader reference, and a certificate-claim field. Leader blocks carry an ancestor list with at most one reference per node and at least $2f+1$ references to round-$(r-1)$ blocks. Only *referenceable* blocks may be used as new hash references: a block is referenceable only when every certificate claim in its causal history is already locally provable from the receiver's current DAG.

This section presents pseudocode for the Bluestreak protocol, adapted from the shared execution skeleton of uncertified DAG BFT protocols in [17].

Algorithm 1: Bluestreak: DAG BFT Execution for Node p_i

```
Global variables and constants:
    δ_LT ← 8Δ                                   // Leader timeout (pull pacemaker)
    n ← 3f + 1                                  // Number of nodes; f is maximum number of Byzantine nodes
    r_decided ← 0                               // Latest round of block in the sequence of decided leaders
    r_highest ← 0                               // Highest round of block in local DAG
    r_last ← 0                                  // Round of latest own block
    DAG ← GenesisBlocks()                       // Local DAG of p_i initialized with predefined blocks of round 0

procedure Execute()                 ◇ Execute the Bluestreak protocol
    for r = 1, 2, ... do
        ExecuteRound(r)

procedure ExecuteRound(r)           ◇ Execute given round
    Step 1:
    τ_enter(r) ← LocalTime()                    // Local time of node p_i
    Broadcast(Advance, r)                       // B1: broadcast round-advance message
    Step 2:
    while TryCreateBlock(r, τ_enter(r)) =false do
        ReceiveMessages()                       // Updates DAG
        TryDecide(r_decided, r_highest)         // Try commit leader blocks and sequence payload
    Step 3:
    r_last ← r
    B ← own round-r block just created
    Broadcast(Block, B)                         // B2: broadcast newly created block
    Step 4:
    while TryAdvanceToNextRound(r)=false do
        ReceiveMessages()                       // Updates DAG
        TryDecide(r_decided, r_highest)         // Try commit leader blocks and sequence payload

function TryAdvanceToNextRound(r)   ◇ Check pacemaker conditions A1 and A2
    if NumberReferenceableInRound(r) ≥ 2f + 1 and r_last = r then
        return true
    return false

function TryCreateBlock(r, τ_enter(r))   ◇ Check pacemaker conditions C1, C2 or C3
    if LeaderConditionsMet(r) or LocalTime() ≥ τ_enter(r) + δ_LT or NumberReferenceableInRound(r) ≥ 2f + 1 then
        CreateBlock(r)                          // See Algorithm 2
        return true
    return false

function LeaderConditionsMet(r)     ◇ Check pacemaker condition C1
    leader_{r−1} ← GetPredefinedLeader(r − 1)
    Leaders ← {B ∈ DAG[r − 1] ∩ Referenceable s.t. B.author = leader_{r−1}}
    L1 ← |Leaders| > 0                          // Referenceable leader block exists
    leader_{r−2} ← GetPredefinedLeader(r − 2)
    Proposals ← {L ∈ DAG[r − 2] s.t. L.author = leader_{r−2}}
    RefVotes ← DAG[r − 1] ∩ Referenceable       // Only referenceable voting blocks
    VoteQuorum ← ∃L ∈ Proposals s.t. |{B.author : B ∈ RefVotes s.t. IsVote(B, L) = true}| ≥ 2f + 1
    // Skip check using only referenceable round-(r−1) blocks
    RefNonVotes ← ∀L ∈ Proposals : |{B.author : B ∈ RefVotes s.t. IsVote(B, L) = false}| ≥ 2f+1
    SkipPattern ← (|Proposals| > 0 and RefNonVotes) or (|Proposals| = 0 and |{B.author : B ∈ RefVotes}| ≥ 2f+1)
    L2 ← VoteQuorum or SkipPattern
    return L1 & L2
```

Algorithm 2: Bluestreak Helper Functions

```
function GetPredefinedLeader(r)                ◇ Return index of leader node for round r
    return (r mod n) + 1

function IsReferenceable(X)                    ◇ Return true if block X can be used as a hash reference by new blocks
    // X is referenceable iff every certificate claim in hist(X) is locally provable
    for B ∈ GetReachableBlocks(X) do
        if B.certClaim ≠ ⊥ then
            L ← the leader block with hash(L) = B.certClaim
            r_L ← L.round
            votes ← |{V.author : V ∈ DAG[r_L + 1] s.t. IsVote(V, L) = true}|
            if votes < 2f + 1 then
                return false                   // Certificate claim not yet locally provable
    return true

function NumberReferenceableInRound(r)         ◇ Return number of distinct nodes with referenceable blocks in round r
    return |{B.author : B ∈ DAG[r] ∩ Referenceable}|

function IsDirectlySkippedSlot(r)              ◇ Return true if leader slot of round r is directly skipped
    leader ← GetPredefinedLeader(r)
    Votes ← DAG[r + 1]                         // All blocks in voting round
    Proposals ← {B ∈ DAG[r] s.t. B.author = leader}   // All proposals for leader slot at round r
    for L ∈ Proposals do
        res ← |{B.author : B ∈ Votes s.t. IsVote(B, L) = false}|
        if res < 2f + 1 then
            return false                       // Found a proposal with fewer than 2f + 1 non-voters
    if |Proposals| = 0 & |{B.author : B ∈ DAG[r + 1]}| < 2f + 1 then
        return false                           // Not enough round-(r + 1) blocks to skip unseen proposals
    return true

function DirectlyCommittedLeaderBlock(r)       ◇ Return directly committed leader block of round r if any
    leader ← GetPredefinedLeader(r)
    Proposals ← {B ∈ DAG[r] s.t. B.author = leader}   // All proposals for leader slot at round r
    BlocksWithClaims ← DAG[r + 2]              // All blocks in certificate-claim round
    for L ∈ Proposals do
        res ← |{B.author : B ∈ BlocksWithClaims s.t. IsCertClaim(B, L) = true}|
        if res ≥ 2f + 1 then
            return L
    return Undecided(r)

function IsVote(B_vote, B_leader)              ◇ Return true if block B_vote votes for leader block B_leader
    return hash(B_leader) ∈ B_vote.ancestors ∪ {B_vote.leader}   // Leader blocks vote via ancestors; non-leaders via the optional leader-ref field

function IsCertClaim(B_cc, B_leader)           ◇ Return true if block B_cc makes a certificate claim for B_leader
    // Non-leader blocks: explicit certificate claim field
    if B_cc.certClaim = hash(B_leader) then
        return true
    // Leader blocks: implicit claim via 2f + 1 round-(r + 1) votes in ancestors
    if B_cc.author ≠ GetPredefinedLeader(B_cc.round) then
        return false                           // Non-leader without matching certClaim field
    r_L ← B_leader.round
    Ancestors ← {B ∈ DAG s.t. hash(B) ∈ B_cc.ancestors and B.round = r_L + 1}
    res ← |{B.author : B ∈ Ancestors s.t. IsVote(B, B_leader) = true}|
    return res ≥ 2f + 1
```

Algorithm 3: Bluestreak Helper Functions (continued)

```
function IsLink(B_old, B_new)          ◇ Return true if B_old is reachable from B_new via references
    return ∃ sequence of blocks B_1, ..., B_k ∈ DAG, k ∈ ℕ, s.t.
        B_1 = B_old, B_k = B_new and
        ∀ j ∈ [2, k] : hash(B_{j-1}) ∈ B_j.ancestors ∪ {B_j.self, B_j.leader}

function GetReachableBlocks(L)         ◇ Return all blocks reachable from leader block L
    return {B ∈ DAG s.t. IsLink(B, L) = true}

function IsCertifiedLink(B_anchor, B_leader)   ◇ Return true if hist(B_anchor) contains a certificate claim for B_leader
    r ← B_leader.round
    BlocksWithClaims ← DAG[r + 2]                                   // All blocks in certificate-claim round
    return ∃B ∈ BlocksWithClaims s.t. IsCertClaim(B, B_leader) = true & IsLink(B, B_anchor) = true

procedure CreateBlock(r)               ◇ Create and sign a round-r block for node p_i
    leader_r ← GetPredefinedLeader(r)
    if p_i = leader_r then
        // Leader block: collect ancestors from referenceable blocks
        PrevRefs ← set of all blocks referenced by any previous block of p_i
        ancestors ← ∅
        for p_j ∈ {p_1, ..., p_n} do
            candidates ← {B ∈ Referenceable \ PrevRefs s.t. B.author = p_j and B.round ≤ r−1}
            if candidates ≠ ∅ then
                B* ← arg max_{B∈candidates} B.round                 // Latest referenceable block of p_j not yet referenced
                ancestors ← ancestors ∪ {hash(B*)}
        // Validate: must include self-ref and ≥ 2f+1 round-(r−1) refs
        B ← (r, ancestors, payload)_{p_i}
    else
        // Non-leader block
        self ← hash(own block from round r−1)
        // Optional leader reference (if referenceable)
        leader_{r−1} ← GetPredefinedLeader(r−1)
        L_cand ← {B ∈ Referenceable s.t. B.author = leader_{r−1} and B.round = r−1}
        leader ← { hash(L)   if L_cand = {L}
                 { ⊥         otherwise
        // Certificate claim for round-(r−2) leader
        leader_{r−2} ← GetPredefinedLeader(r−2)
        Proposals ← {L ∈ DAG[r−2] s.t. L.author = leader_{r−2}}
        certClaim ← ⊥
        for L ∈ Proposals do
            votes ← |{V.author : V ∈ DAG[r−1] s.t. IsVote(V, L) = true}|
            if votes ≥ 2f+1 then
                certClaim ← hash(L)                                 // Observed certificate pattern for L
        B ← (r, {self, leader}, certClaim, payload)_{p_i}
    DAG ← DAG ∪ {B}
    Referenceable ← Referenceable ∪ {B}                             // Own block is immediately referenceable
```

Algorithm 4: Bluestreak Leader Committer and Payload Sequencer

```
Global variables and constants:
    PayloadSeq ← [ ]                                   // Global output sequence of payloads
    Sequenced ← ∅                                      // Set of (author, round) pairs already appended to PayloadSeq

procedure TryDecide(r_decided, r_highest)              ◇ Update sequence of commit/skip decisions for leaders
    sequence ← [ ]
    for r ∈ [r_highest − 1 down to r_decided + 1] do
        status ← TryDirectDecide(r)
        if status = Undecided(r) then
            status ← TryIndirectDecide(r, sequence)
        sequence ← status||sequence
    for status ∈ sequence in ascending round-order do
        if status = Undecided(r) then
            break
        if status = Commit(L) then
            SequencePayload(L)
        r_decided ← r_decided + 1                      // Update last decided round

function TryDirectDecide(r)                            ◇ Return direct commit/skip decision or undecided state
    if IsDirectlySkippedSlot(r) = true then
        return Skip(r)
    L ← DirectlyCommittedLeaderBlock(r)
    if L ≠ Undecided(r) then
        return Commit(L)
    return Undecided(r)

function TryIndirectDecide(r, sequence)                ◇ Return indirect commit/skip decision or undecided state
    Anchors ← {s ∈ sequence s.t. s.round ≥ r + 3}
    for A ∈ Anchors in ascending round-order do
        if A is Undecided then
            return Undecided(r)                        // Cannot resolve: anchor still undecided
        if A = Commit(B_anchor) then
            leader ← GetPredefinedLeader(r)
            Proposals ← {B ∈ DAG[r] s.t. B.author = leader}   // All proposals for leader slot at round r
            for L ∈ Proposals do
                if IsCertifiedLink(B_anchor, L) then
                    return Commit(L)
            return Skip(r)
    return Undecided(r)

procedure SequencePayload(L)                           ◇ Sequence payloads from committed leader block
    Reachable ← sort(GetReachableBlocks(L))
    for B ∈ Reachable do
        if (B.author, B.round) ∉ Sequenced then
            Sequenced ← Sequenced ∪ {(B.author, B.round)}   // Ignore equivocations
            PayloadSeq ← PayloadSeq||[B.payload]
```

Algorithm 5: Bluestreak Dissemination and Pull Recovery for Node p_i

```
Global variables and constants:
    ∀j : Known[j] ← GenesisBlocks()                          // Genesis blocks are globally known
    Suspended ← ∅                                            // Buffer of verified blocks whose referenced history is not yet in DAG
    Referenceable ← GenesisBlocks()                          // Set of blocks whose certificate claims are all locally provable

procedure Broadcast(tag, msg)          ◇ Broadcast tagged message to all peers
    for j ∈ {1, ..., n} \ {i} do
        Send(p_j, tag, msg)                                  // Receiver gets (tag, sender, msg)

// Verify(B): check signature validity and block-format structural requirements (Section E.1)

procedure ReceiveMessages()            ◇ Receive blocks or advance messages and update local state
    (tag, sender, msg) ← ReceiveNetworkMessage()
    if tag = Block then
        B ← msg
        if Verify(B) = true then
            AddToDAG(B)
            if B ∈ Suspended or (B ∈ DAG and B ∉ Referenceable) then
                StartPullTimer(sender, B, δ_D)               // P1: causal or referenceability prerequisites missing
    if tag = Advance then
        r_adv ← msg
        if current round < r_adv then
            StartPullTimer(sender, r_adv, δ_A)               // P1: behind on rounds
    if tag = PullRequest then
        response ← GatherMissingPrereqs(msg, sender)
        Send(sender, PullResponse, response)
        Known[sender] ← Known[sender] ∪ response             // Sender now has these blocks
    if tag = PullResponse then
        for B ∈ msg do
            if Verify(B) = true then
                AddToDAG(B)

procedure StartPullTimer(sender, target, delay)   ◇ Start a pull-delay timer (P1); pull on expiry if still needed (P2)
    Schedule after delay:
    if prerequisites for target still missing then
        Send(sender, PullRequest, target)

function GatherMissingPrereqs(target, requester)   ◇ Collect blocks the requester needs for target
    // For a block target: return referenced history and voting blocks needed for referenceability
    // For a round target: return 2f+1 referenceable blocks per missing round, plus their history
    result ← ∅
    needed ← blocks in DAG that target transitively requires
    for B ∈ needed do
        if B ∉ Known[requester] then
            result ← result ∪ {B}
    return result

function CausalAvailable(B)            ◇ Return true iff all blocks referenced by B are already in DAG
    refs ← {B.self}
    if B is a leader block then
        refs ← B.ancestors
    if B.leader ≠ ⊥ then
        refs ← refs ∪ {B.leader}
    return ∀h ∈ refs : ∃B' ∈ DAG s.t. hash(B') = h

procedure AddToDAG(B)                  ◇ Integrate block into local DAG subject to causal availability
    Suspended ← Suspended ∪ {B}
    while ∃B' ∈ Suspended s.t. CausalAvailable(B') = true do
        B' ← any such block in Suspended
        Suspended ← Suspended \ {B'}
        DAG ← DAG ∪ {B'}
        r_highest ← max(r_highest, B'.round)
        Known[B'.author] ← Known[B'.author] ∪ GetReachableBlocks(B')   // Author knew this history
        UpdateReferenceability(B')                           // New block may provide voting evidence for pending claims

procedure UpdateReferenceability(B_new)   ◇ Re-check referenceability of DAG blocks after B_new is added
    // Adding B_new may supply a vote that makes some certificate claim locally provable,
    // which in turn makes blocks carrying that claim referenceable
    for X ∈ DAG \ Referenceable do
        if IsReferenceable(X) = true then
            Referenceable ← Referenceable ∪ {X}
```